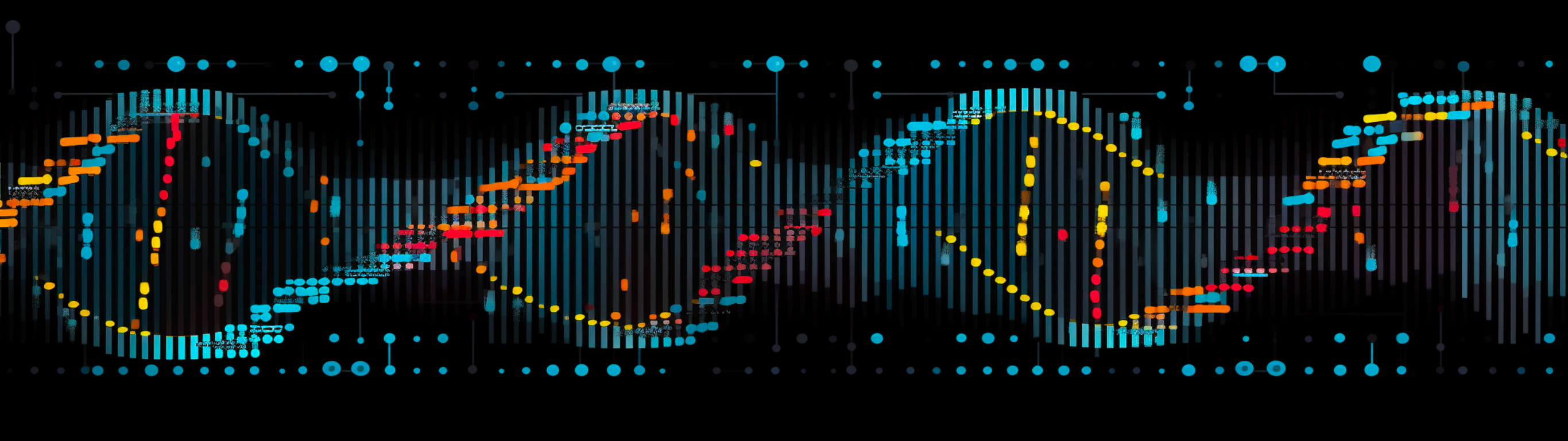




Introduction to bioinformatics

Chapter4 Machine learning in bioinformatics
- Network and clustering - Part3
PPI and clustering

Jiaxing Chen

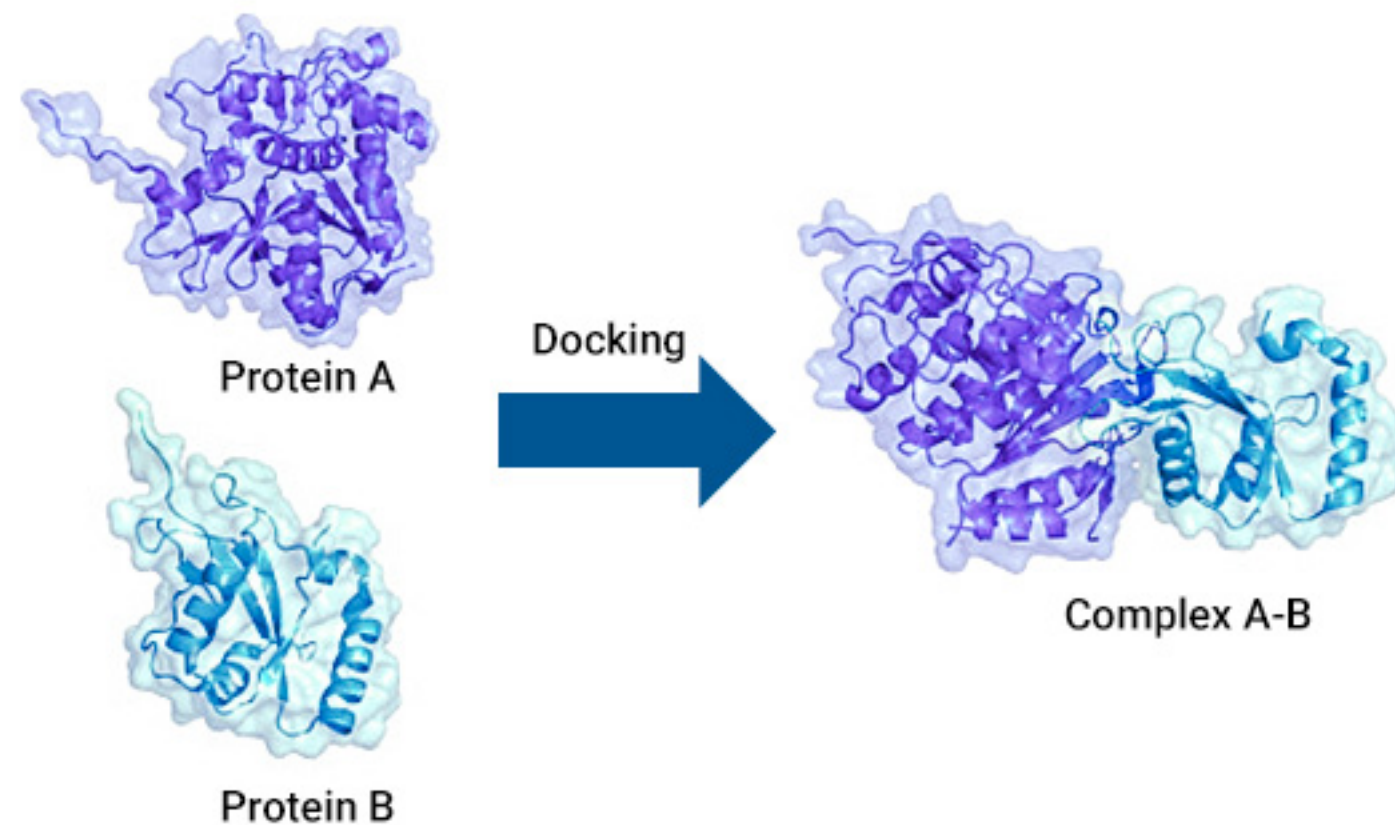
Outline

- **PPI and protein representation**
- Similarity and Dissimilarity
- Clustering / community detection
 - WGCNA and scale-free
 - Modularity and Louvain

Protein Protein Interaction

Protein-protein interactions refer to specific and intentional physical contacts established between two or more proteins to carry out specific biological activity.

Protein-protein interactions play a fundamental role in nearly all cellular processes, including enzymatic reactions, antibody-antigen complexes, large biomolecular assemblies, signal transduction, intercellular communication, metabolic pathways, transport, gene expression, and cell growth and proliferation.



Transfer learning via multi-scale convolutional neural layers for human–virus protein–protein interaction prediction

Bioinformatics 2021

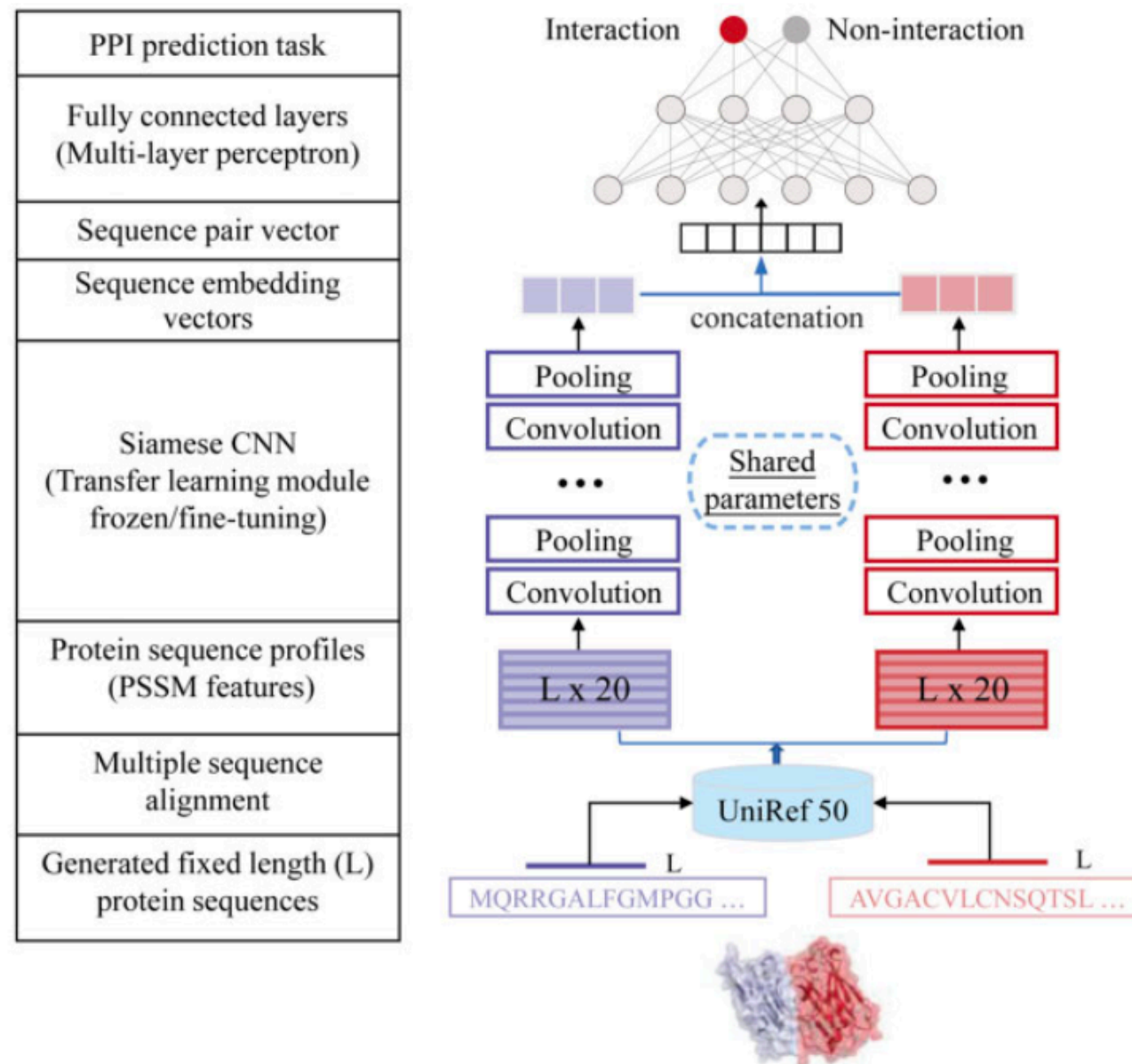
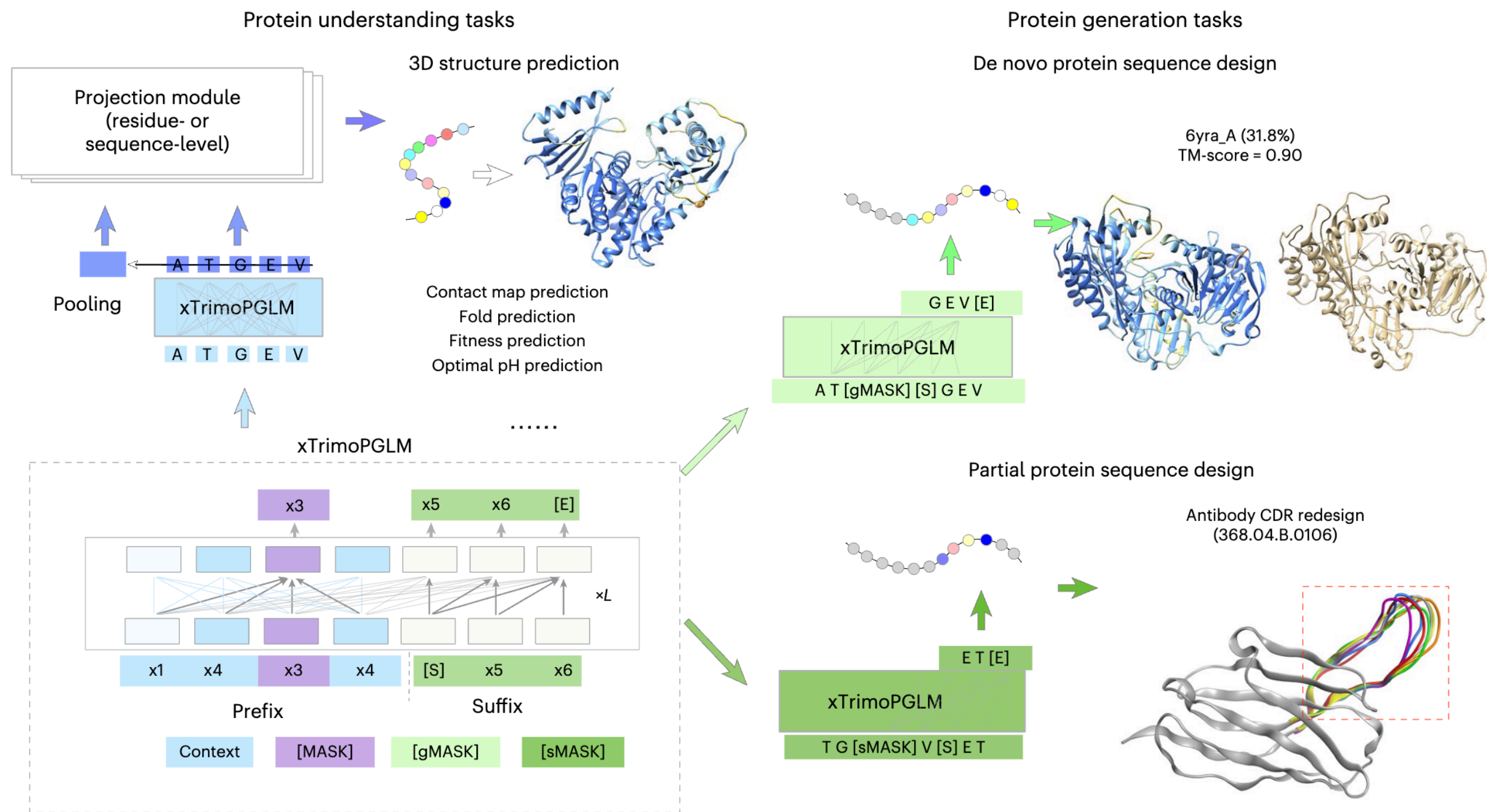


Fig. 1. Our proposed deep learning architecture to predict human–virus PPIs combines evolutionary sequence profile features of interacting human and viral proteins with a Siamese CNN architecture and an MLP

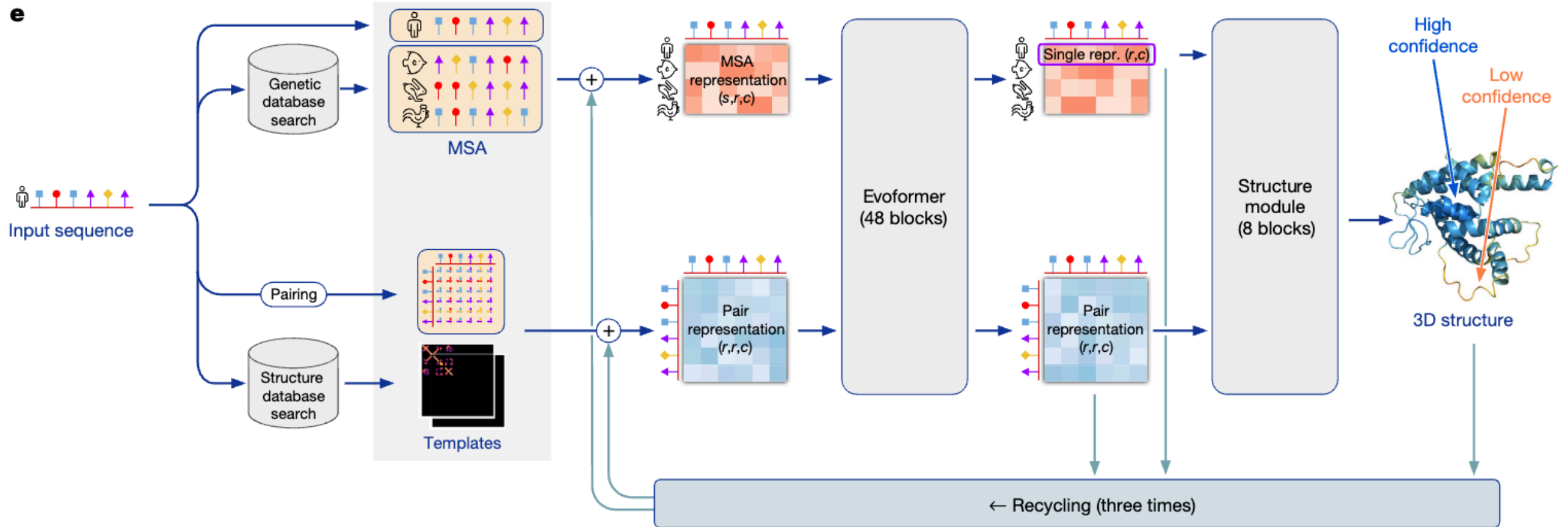
xTrimoPGLM: unified 100-billion-parameter pretrained transformer for deciphering the language of proteins

Nature method 2025



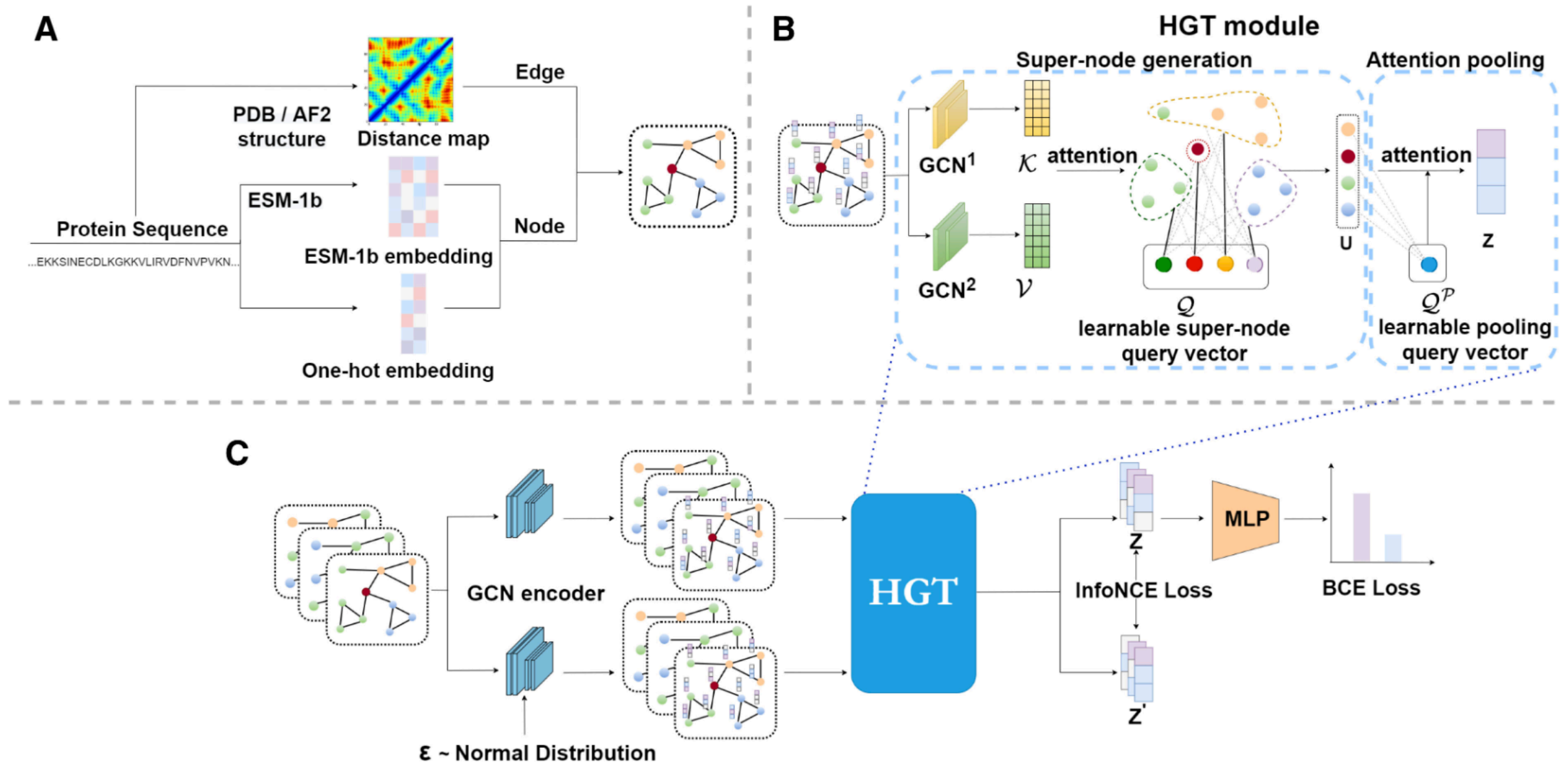
Our key technical contribution is an exploration of the compatibility and the potential for joint optimization of the two types of objectives, which has led to a strategy for training xTrimoPGLM at an unprecedented scale of 100 billion parameters and 1 trillion training tokens. Our extensive experiments reveal that (1) xTrimoPGLM substantially outperforms other advanced baselines in 18 protein understanding benchmarks across four categories. The model also facilitates an atomic-resolution view of protein structures, leading to an advanced three-dimensional structural prediction model that surpasses existing language model-based tools.

Highly accurate protein structure prediction with AlphaFold



Hierarchical graph transformer with contrastive learning for protein function prediction

Bioinformatics 2023



EQUIFORMER: EQUIVARIANT GRAPH ATTENTION TRANSFORMER FOR 3D ATOMISTIC GRAPHS
ICLR 2023

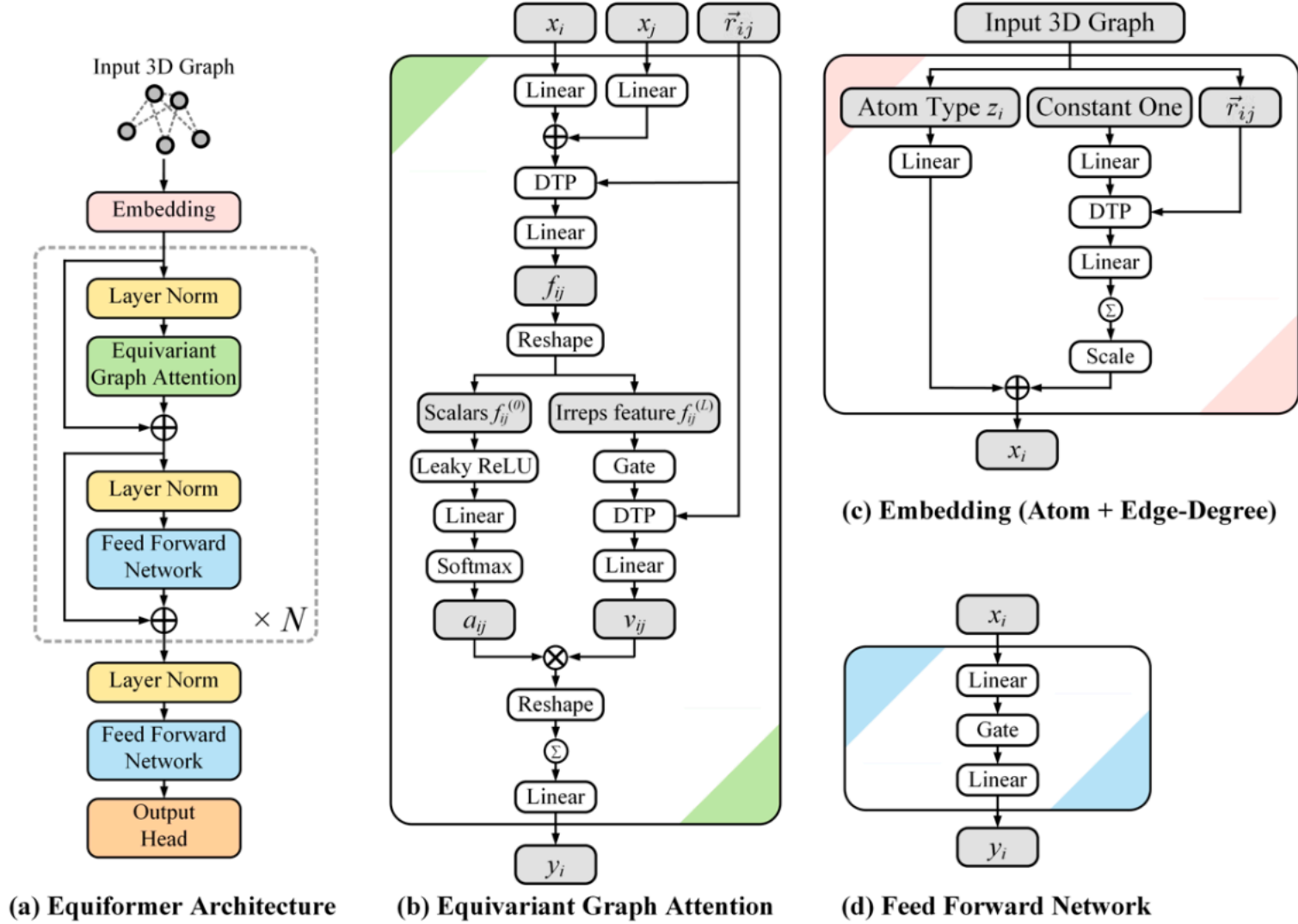


Figure 1: Architecture of Equiformer. We embed input 3D graphs with atom and edge-degree embeddings and process them with Transformer blocks, consisting of equivariant graph attention and feed forward networks. In this figure, “ $\otimes$ ” denotes multiplication, “ $\oplus$ ” denotes addition, and “DTP” stands for depth-wise tensor product. $\sum$ within a circle denotes summation over all neighbors. Gray cells indicate intermediate irreps features.

Multi-Scale Representation Learning on Proteins
NeurIPS 2021

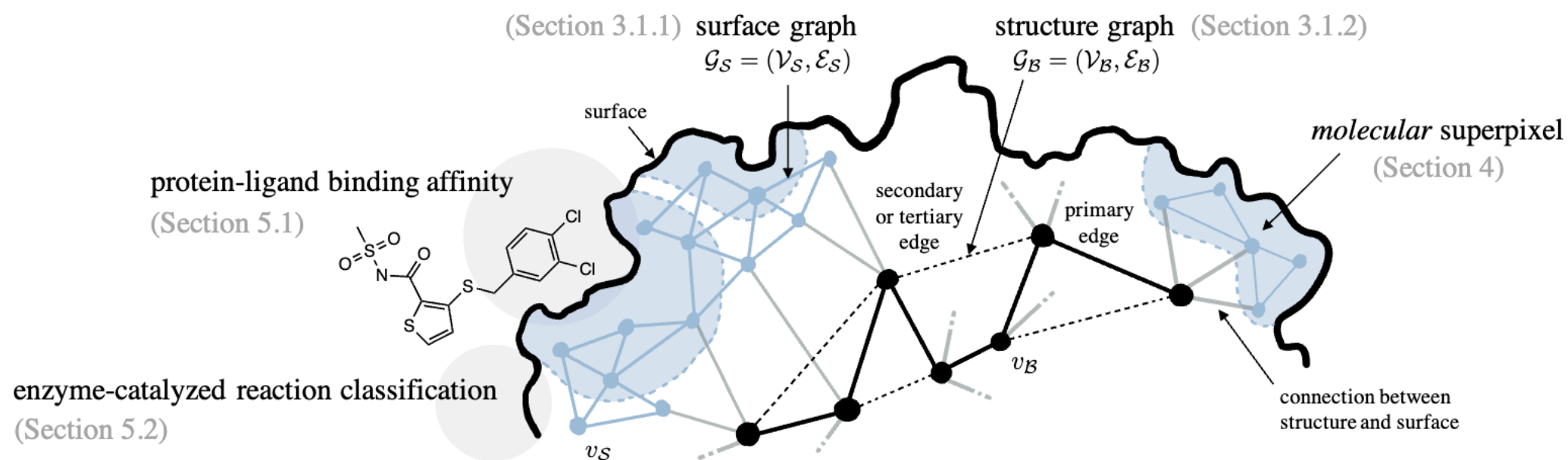


Figure 1: Overview of HOLOPROT Our multi-scale protein representation algorithm integrates primary, secondary and tertiary elements of protein structures and connects them to the surface. We extract higher-level protein motifs by introducing *molecular* superpixels. Both structure and surface are represented as graphs $\mathcal{G}_B$ and $\mathcal{G}_S$, respectively. The method is evaluated on two representative tasks, protein-ligand binding affinity and enzyme-catalyzed reaction classification.

ScanNet: an interpretable geometric deep learning model for structure-based protein binding site prediction Nature method 2022

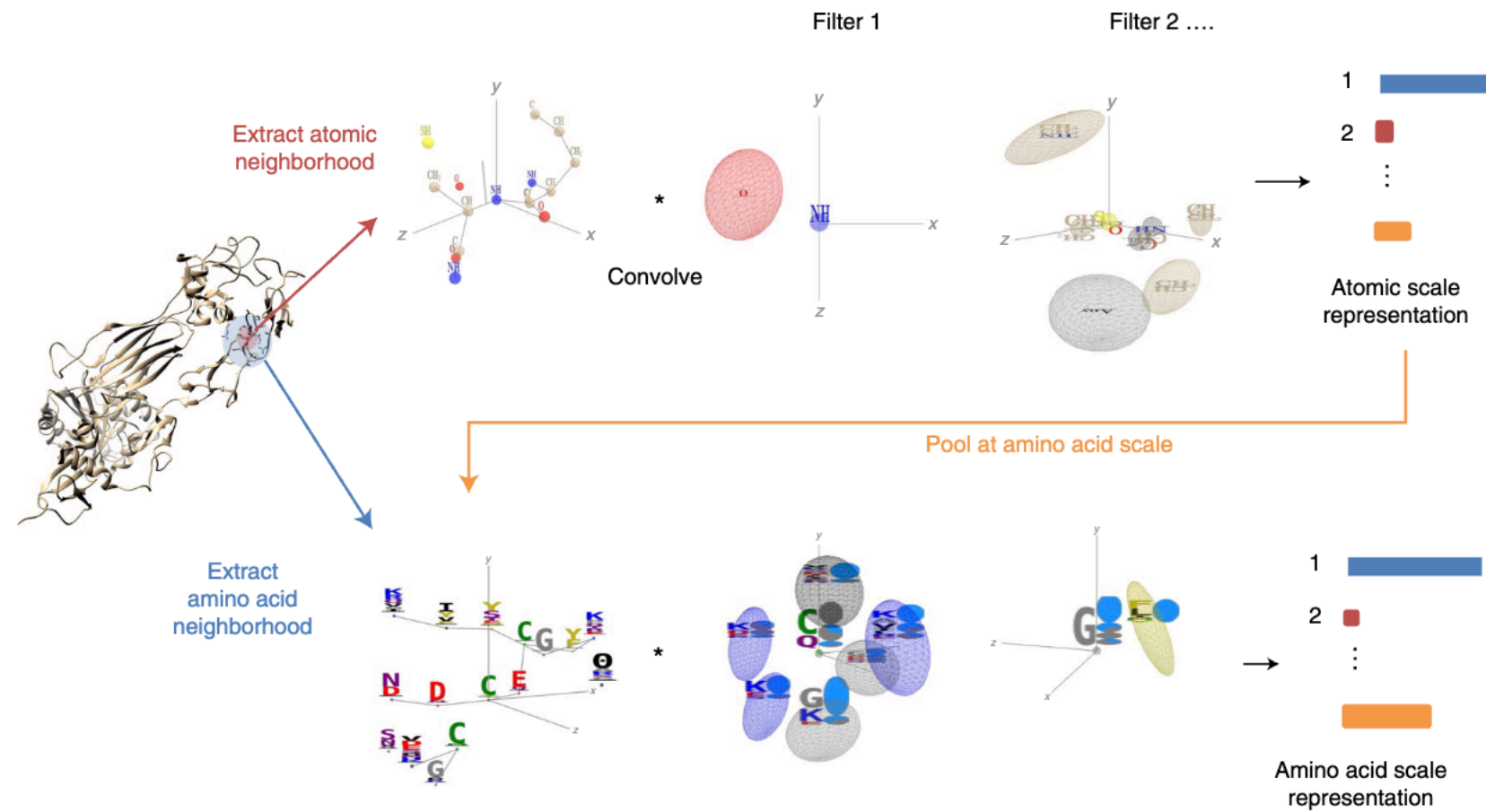
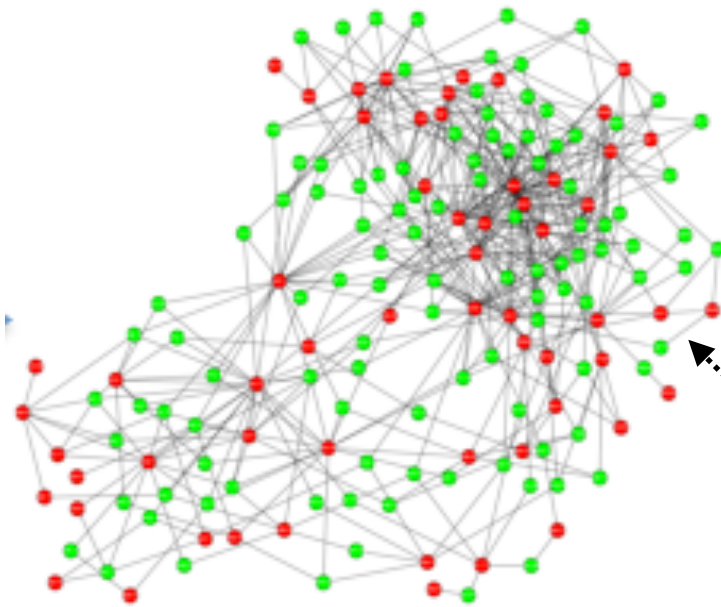


Fig. 1 | Overview of the ScanNet architecture. ScanNet inputs are the primary sequence, tertiary structure and, optionally, position-weight matrix computed from a MSA of evolutionarily related proteins. First, for each atom, neighboring atoms are extracted from the structure and positioned in a local coordinate frame (top left). The resulting point cloud is passed through a set of trainable, linear filters detecting specific spatio-chemical arrangements (top middle), yielding an atomic-scale representation (top right). After aggregation of the atomic representation at the amino acid level and concatenation with amino acid attributes, the process is reiterated with amino acids to obtain a representation of an amino acid (bottom). The latter is projected and locally averaged for residue-wise classification.

Outline

- PPI and protein representation
- **Similarity and Dissimilarity**
- Clustering / community detection
 - WGCNA and scale-free
 - Modularity and Louvain

Weighted graph



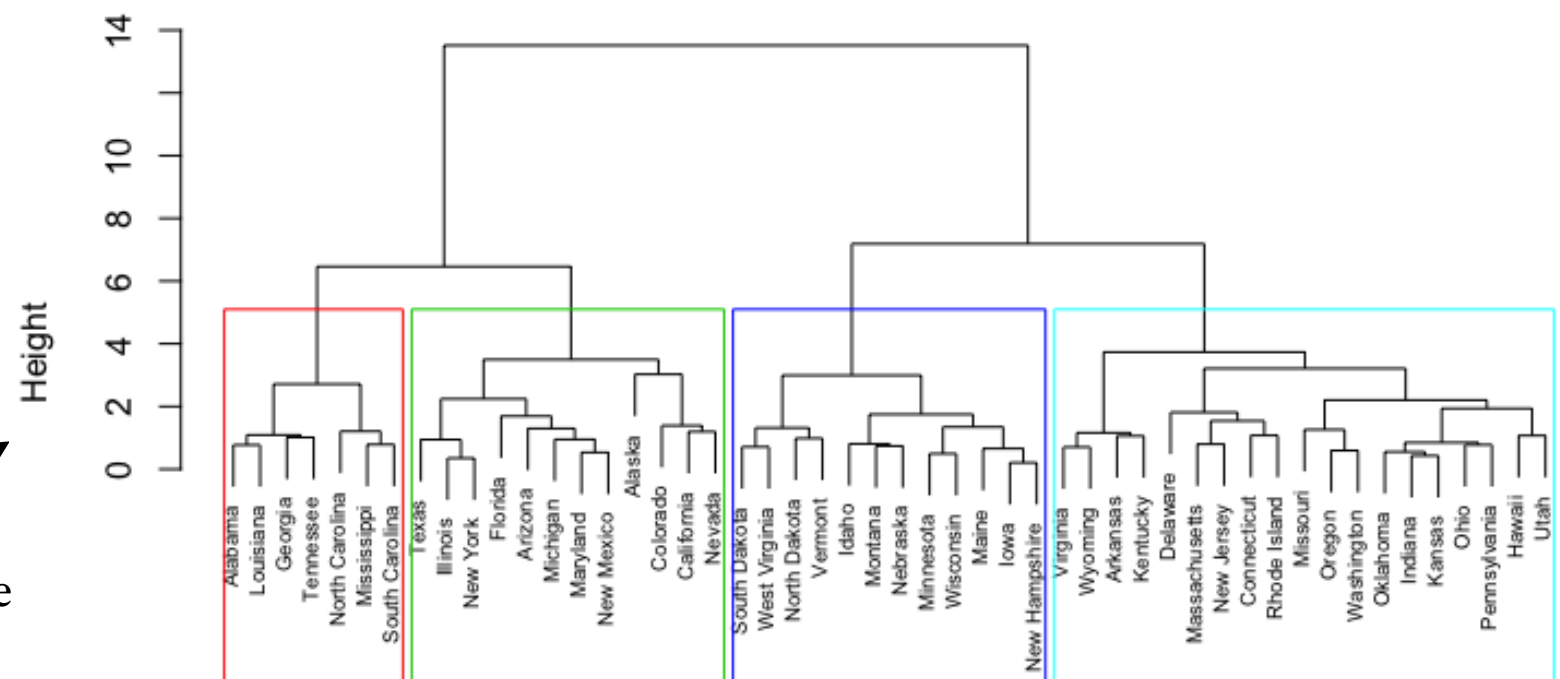
Similarity



Dissimilarity

Cluster Dendrogram

Height~ Dissimilarity/Distance



d
hclust (*, "ward.D2")

Distance/Dissimilarity & Similarity

- Let d_{ij} denote the **distance/dissimilarity** between two objects x_i and x_j
 - The objects are, for example, **strings**, **sequences**, **structures**, **words**, **documents**, **pixels**, or **vectors** (of features)
- Similarly s_{ij} denotes the **similarity** between x_i and x_j
- Comparing some objects are better done with a similarity measure, while comparing some other objects are better with a dissimilarity measure

Desirable properties

- Conditions for metric distance
 - $d_{ij} \geq 0$ (non-negativity)
 - $d_{ij} = 0$ if and only if $i = j$ (identity of indiscernible pairs)
 - $d_{ij} = d_{ji}$ (symmetry)
 - $d_{ij} \leq d_{ik} + d_{kj}$ (triangular inequality)
- Similar intuitions for other dissimilarity (or similarity) measures
- Many dissimilarity/similarity measures can be defined

Examples of dissimilarity measures

□ Strings/Sequences

- Hamming distance
- Sequence alignment, e.g. edit distance

□ Structure

- Root Mean Square Deviation (RMSD)
- See https://en.wikipedia.org/wiki/Structural_alignment

Hamming distance, edit distance, RMSD are all metric

□ Vectors

- Euclidean distance
- Metric distance
- Non-metric distance

Examples of similarity measures

□ Words/Documents

- Bag-of-words, tf-idf
- Semantic (https://en.wikipedia.org/wiki/Semantic_similarity)
- Vector (https://en.wikipedia.org/wiki/Word_embedding)

□ Vectors

- Correlations (Pearson *etc.*)
- Covariance
 - Principal Component Analysis
- Gaussian $e^{-\|x_i - x_j\|^2 / 2\sigma^2}$
 - Mapping to infinite dimensional space (Kernel function)
 - Probability distribution (co-occurrence probability)
 - Heat function (transition probability)

Gaussian function

- The Gaussian function is

$$K(x_i, x_j) = e^{-\|x_i - x_j\|^2 / 2\sigma^2}$$

- Used prominently in
 - Kernel methods
 - Image segmentation (Wu and Leary 1993, Normalized Cut 1997)
 - Dimensionality reduction (Eigenmap 2003, Diffusion maps 2005, t-SNE 2007, UMAP 2018)
- Pros:
 - Linear combination of $(x_i^\top x_j)^k$ terms for all powers of k
 - Fast decay to zero
 - Symmetric, non-negative, identity
- Con: Sensitive to σ

Converting $d_{ij} \Leftrightarrow s_{ij}$

- Difficult to obtain s_{ij} from d_{ij} and vice versa
 - Most conversions will be dissatisfactory, resulting in non-metric distance
- **Ad hoc conversion** between dissimilarity $D = (d_{ij})$ and similarity $S = (s_{ij})$
 - Inverse conversion
 - $d_{ij} = \text{const} * (1 + s_{ij})^{-1}$
 - $s_{ij} = \text{const} * (1 + d_{ij})^{-1}$
 - Linear conversion
 - $d_{ij} = \text{const} - s_{ij}$
 - $s_{ij} = \text{const} - d_{ij}$

Euclidean distance $d_{ij} \Leftrightarrow s_{ij}$

- Let $D = (d_{ij})$ be given by the Pythagorean

$$d_{ij}^2 = (x_i - x_j)(x_i - x_j)^\top$$

where x_i and x_j are row vectors

- For $S = (s_{ij})$

- Cosine similarity

$$s_{ij} = \frac{x_i x_j^\top}{\|x_i\| \|x_j\|}$$

- Linear kernel similarity

$$s_{ij} = x_i x_j^\top$$

- Con: $s_{ij} \leq s_{uv}$ does not imply $d_{ij} \geq d_{uv}$
- Pro: Can be converted to d_{jk} easily (next slide)

Euclidean distance $d_{ij} \Leftrightarrow s_{ij}$

- If s_{ij} is the linear kernel similarity, that is,

$$s_{ij} = x_i x_j^\top$$

- $d_{ij}^2 = (s_{ii} + s_{jj}) - 2s_{ij}$

- $S = -\frac{1}{2}CDC$

where

$C = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, the centering matrix

$\mathbf{1}$ is a column vector of all ones (hence $\mathbf{1}\mathbf{1}^\top$ is a matrix with all ones of the same dimension as D)

- No similar relation exists for the cosine distance (use ad hoc)

Gaussian similarity $s_{ij} \Leftrightarrow d_{ij}$

- For Gaussian similarity $S = (s_{ij})$ and dissimilarity $D = (d_{ij})$
 - $s_{ij} = e^{-\frac{d_{ij}^2}{2\sigma^2}}$
 - Intuitively $d_{ij} = -\alpha \log(s_{ij})$
 - $d_{ii} = 0$ and $d_{ij} = d_{ji}$
 - Alternatively, define an induced distance $d'_{ij} = s_{ii} + s_{jj} - 2s_{ij}$, then
 - $d'_{ij} = 2(1 - s_{ij})$
 - $d'_{ii} = 0$
 - $d'_{ij} = d'_{ji}$

But still no triangular inequality guarantee

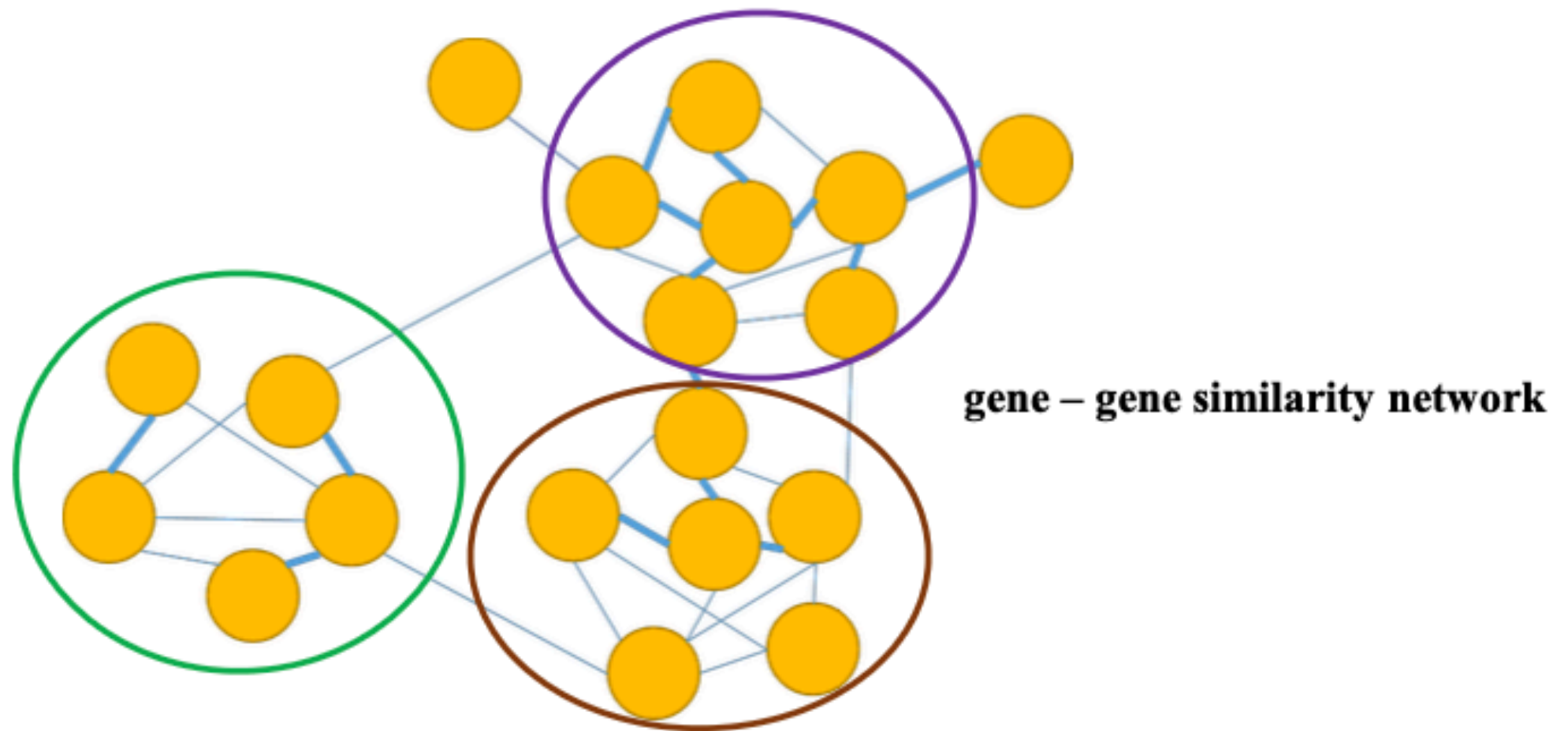
Outline

- **PPI and protein representation**
- Similarity and Dissimilarity
- Clustering / community detection
 - **WGCNA and scale-free**
 - Modularity and Louvain

WGCNA: an R package for weighted correlation network analysis

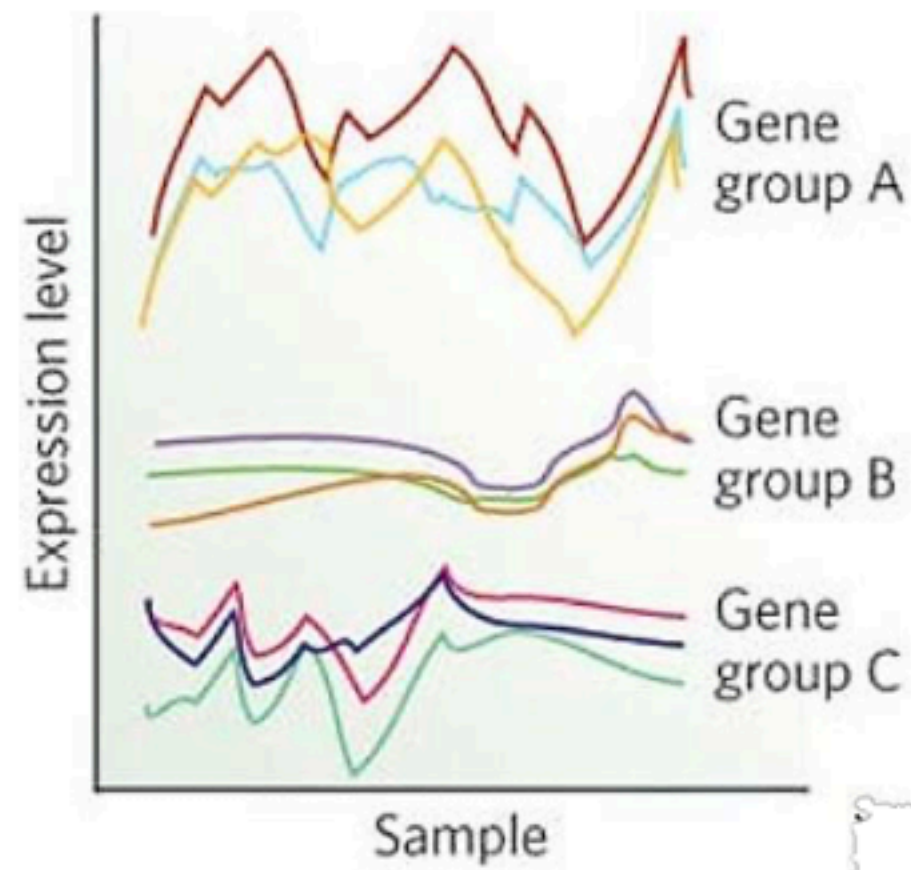
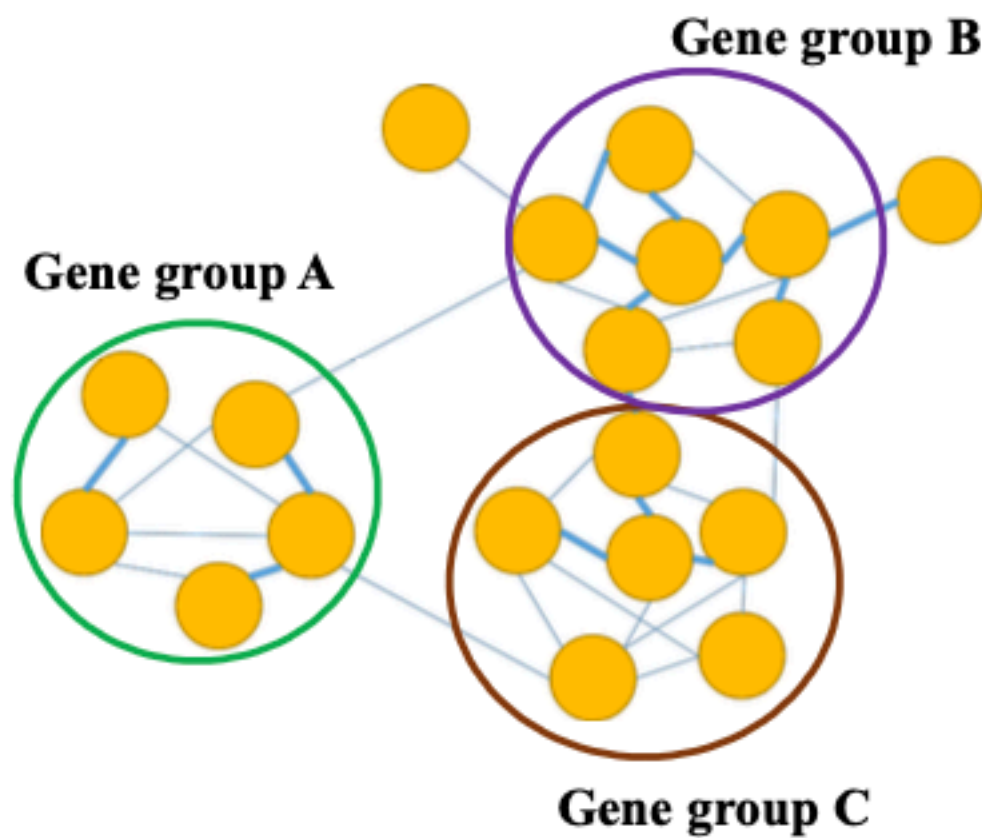
Introduction

weighted correlation network:



Introduction

Identify modules :



[Daniel H. Geschwind & Genevieve Konopka. Neuroscience in the era of functional genomics and systems biology, Nature 461, 908-915]

Construct weighted correlation network

Construct adjacency matrix: $\rightarrow$ *adjacency()* function

$$a_{i,j} = |cor(i,j)|^\beta, \beta = \text{soft threshold}$$

Example: $\beta = 4$

$$\begin{array}{ll} cor(i,j) = 0.8 & \rightarrow |0.8|^4 = 0.4096 \\ cor(k,l) = 0.2 & \rightarrow |0.2|^4 = 0.0016 \end{array}$$

Strong correlation

weak correlation

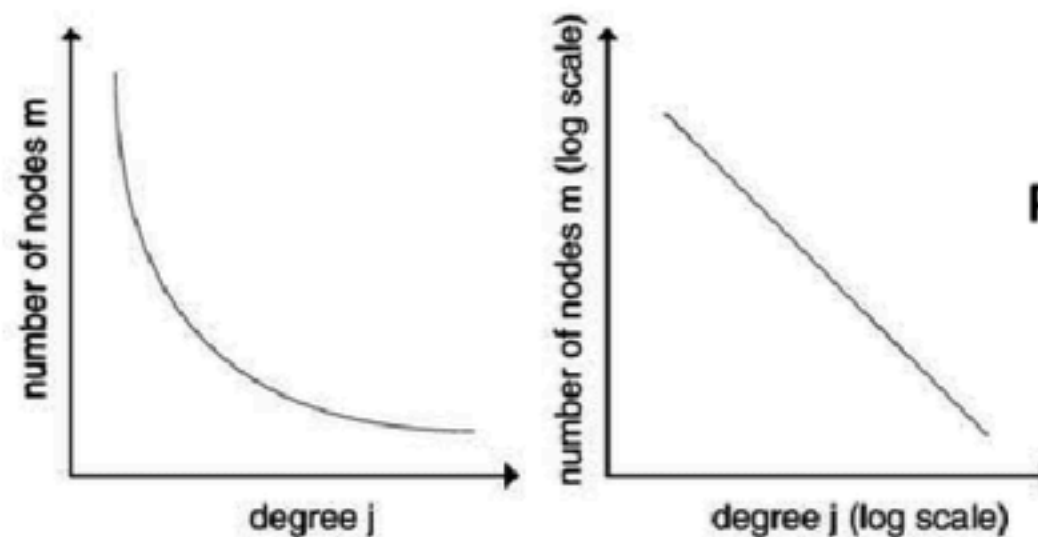
Construct weighted correlation network

How Choose Best β value?



Objective function:

Pick lowest possible that leads to an approximately
scale-free network topology



Power law function: $p_{deg}(k) \propto k^{-\gamma}$

[https://www.researchgate.net/figure/Scale-Free-Network-Left-Power-Law-Degree-Distribution-curve-Right-on-log-log-scale_fig1_310261624]
[<https://bmcbioinformatics.biomedcentral.com/articles/10.1186/1471-2105-9-559>]

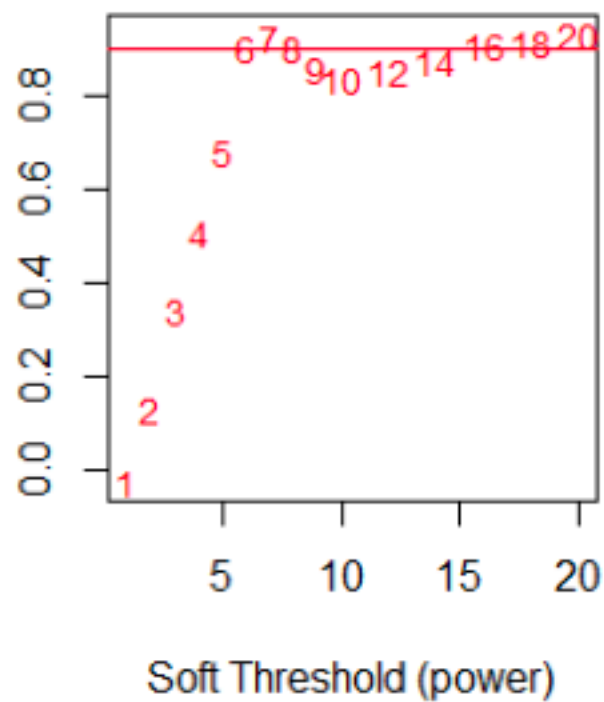
Construct weighted correlation network

Choose β value

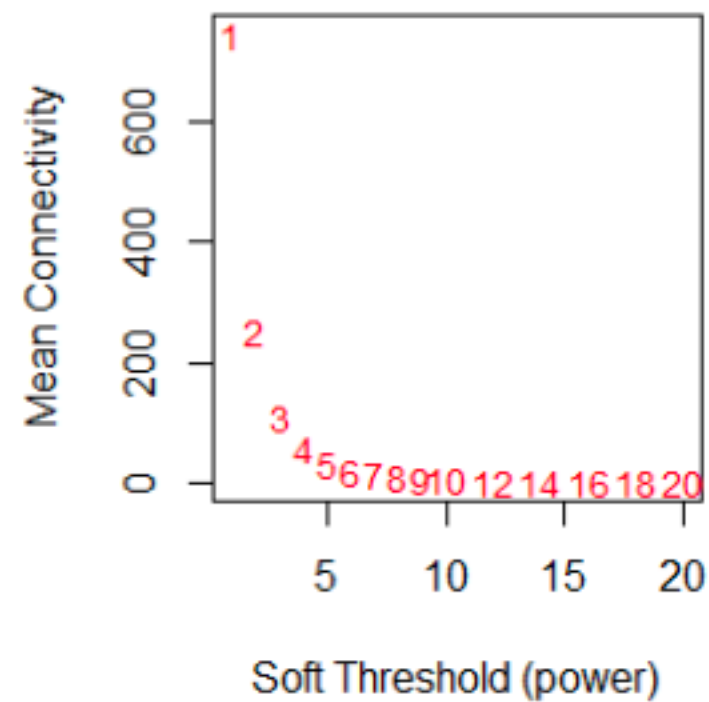
→ *pickSoftThreshold()* function

Scale Free Topology Model Fit, signed R^2

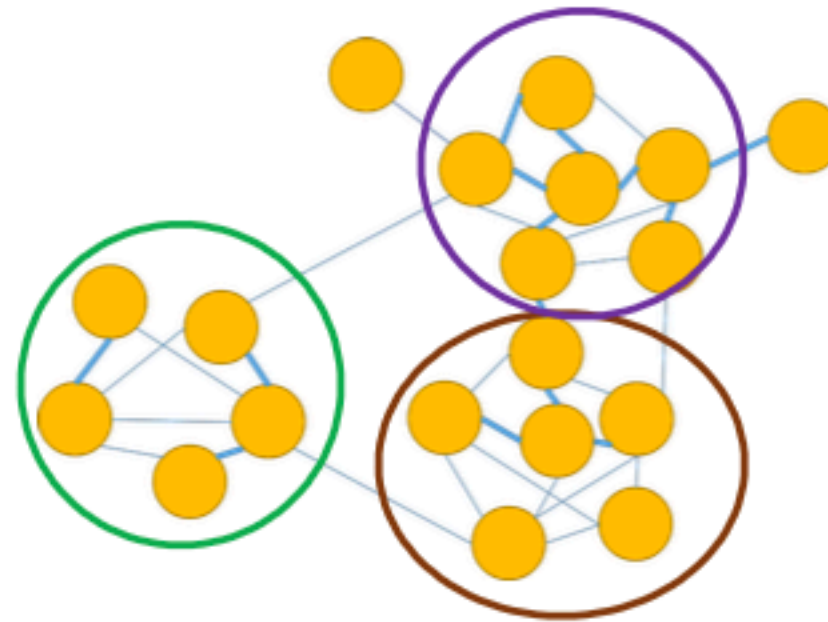
Scale independence



Mean connectivity



Identify modules



Modules = co-express genes

✓ Topological Overlap Measure(TOM) = Similarity between genes

Identify modules

Compute dissimilarity between genes:

Topological Overlap Measure (TOM) matrix:

→ *TOMsimilarity(adjacency) function*

→ *TOMsimilarityFromExpr (Expr , power)*

$$Tom(i,j) = \frac{|N_i \cap N_j| + \alpha_{ij}}{\min(|N_i|, |N_j|) + 1 - \alpha_{ij}}$$

$$dissimilarity_{ij} = 1 - Tom_{ij}$$

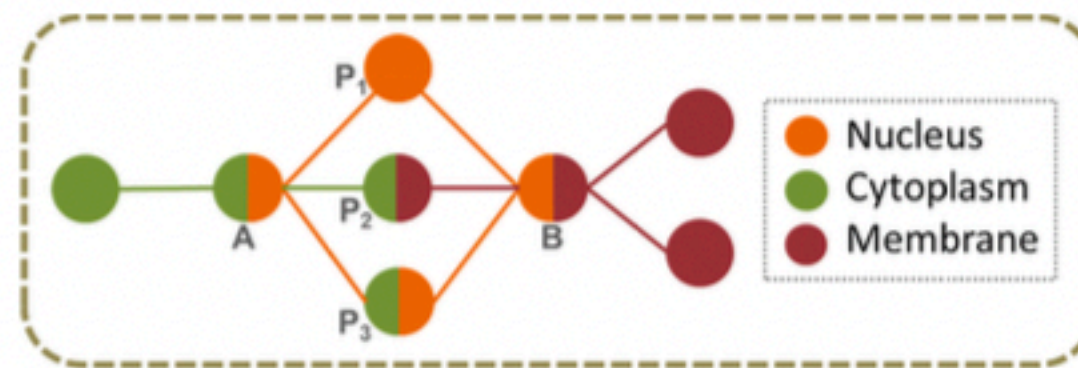
$$\alpha_{ij} \uparrow: Tom(i,j) \uparrow, \quad dissimilarity \downarrow$$

[https://www.researchgate.net/post/What_do_adjacency_matrix_and_Topology_Overlap_Matrix_from_WGCNA_package_tell_about_the_data]

Identify modules

Compute dissimilarity between genes:

Topological Overlap Measure (TOM) matrix:



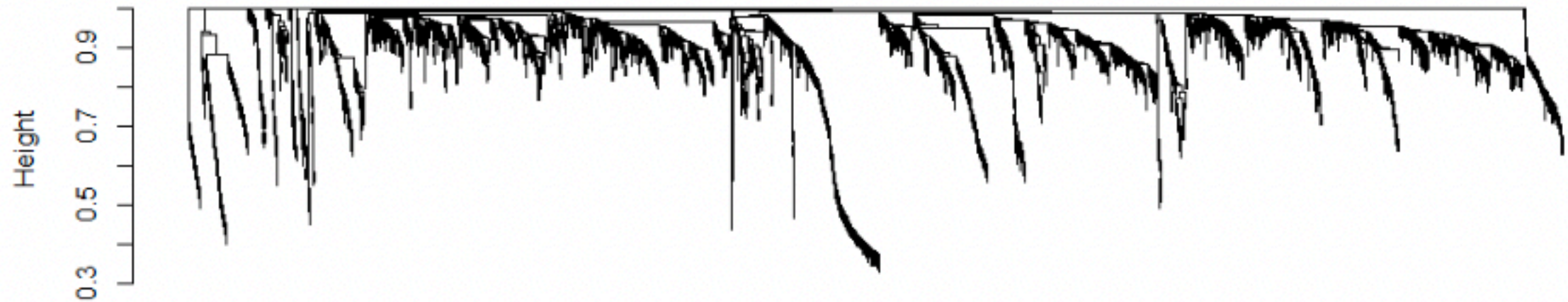
[<https://link.springer.com/article/10.3786/s12859-019-2598-7>]

$$\begin{cases} \deg_A = 4 \\ \deg_B = 5 \\ |N_A \cap N_B| = 3 \\ \alpha_{AB} = 0.5 \end{cases} \rightarrow TOM(A, B) = \frac{3 + 0.5}{4 + 1 - 0.5} = 0.77, \text{dissimilarity} = 0.23$$

Identify modules

Perform hierarchical clustering of genes

Gene clustering on TOM-based dissimilarity



Identify modules

Merge very similar modules

✓ Calculate eigengene

- **Example:** *module z* eigengene

	genA	genB	gen C		gen N
S1					
S2					
S3					
.....					
S123					

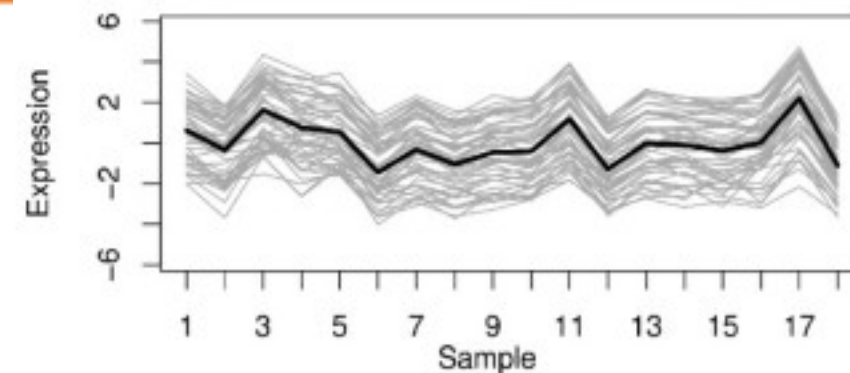
PCA



***Module z* eigengene**

	PC 1
S1	
S2	
S3	
.....	
S123	

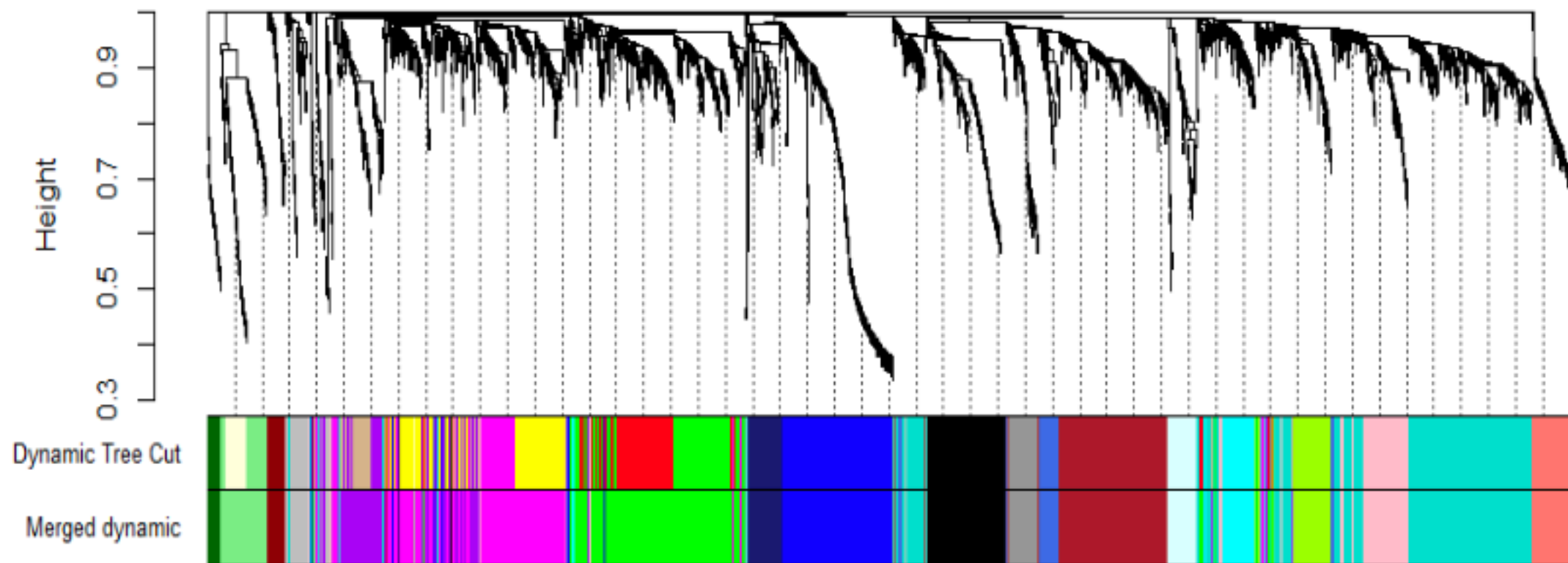
C. Expression levels of genes and eigengene



Identify modules

Merge very similar modules

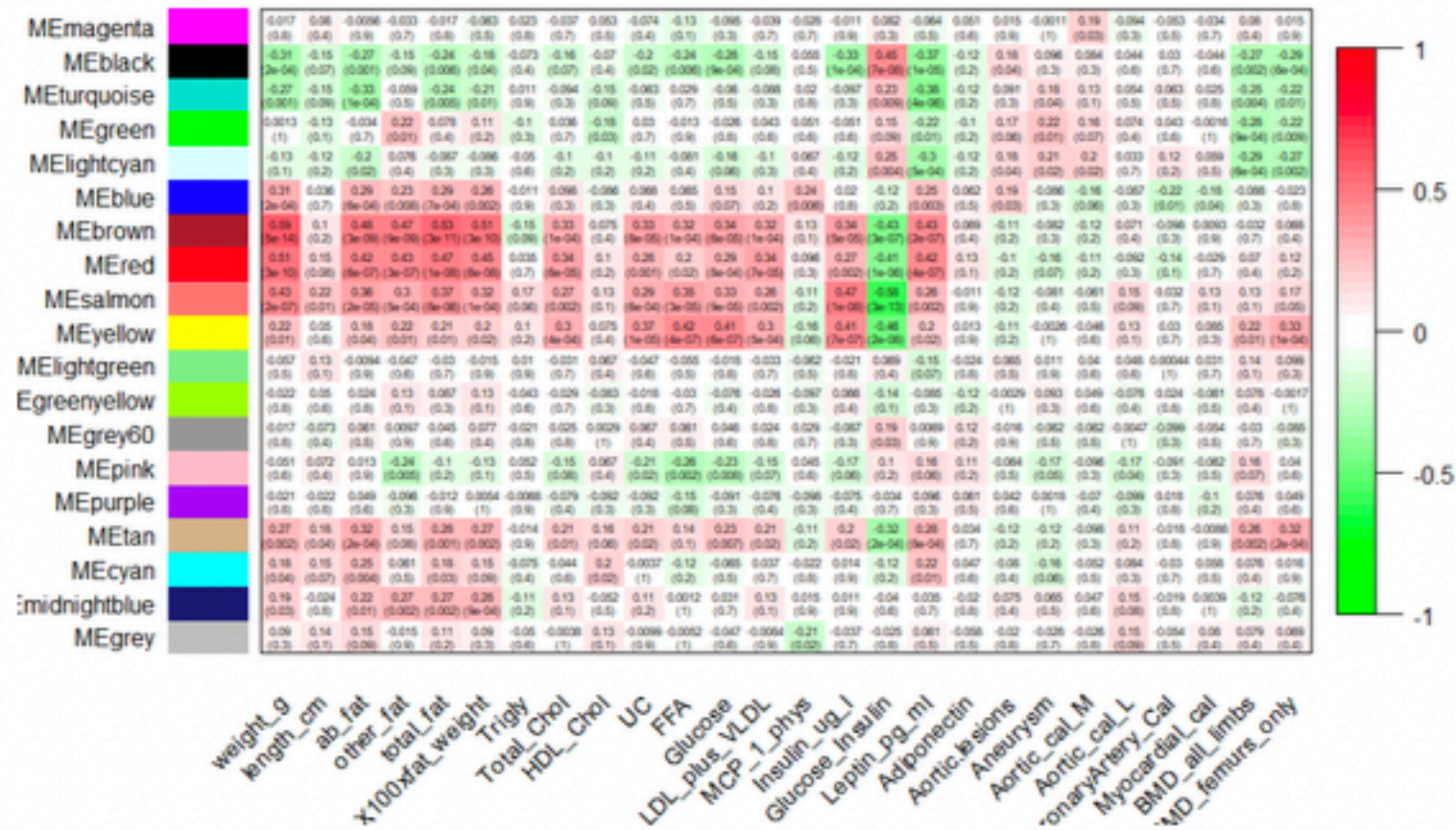
✓ Merge modules → *mergeCloseModules()* function



Relate modules to external information

Compute correlation:
 $\text{cor}(\text{MES}, \text{external traits})$

Module-trait relationships



Outline

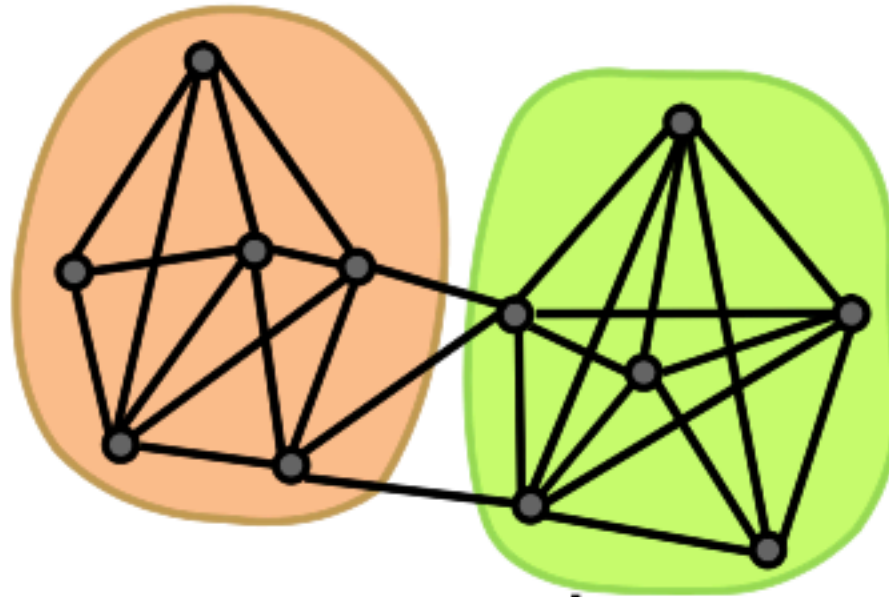
- Protein Protein Affinity
- Similarity and Dissimilarity
- Clustering / community detection
 - WGCNA and scale-free
 - **Modularity and Louvain**

Networks & Communities

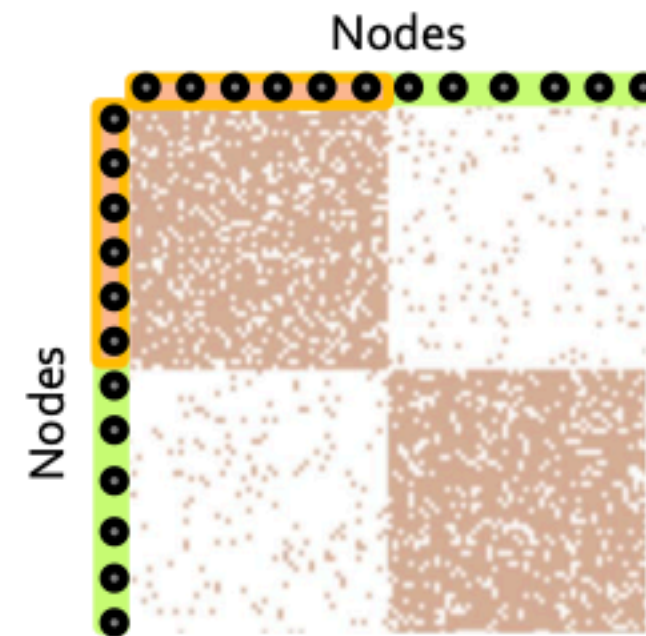
- We often think of networks being organized into **modules, clusters, communities**:



Non-overlapping Clusters

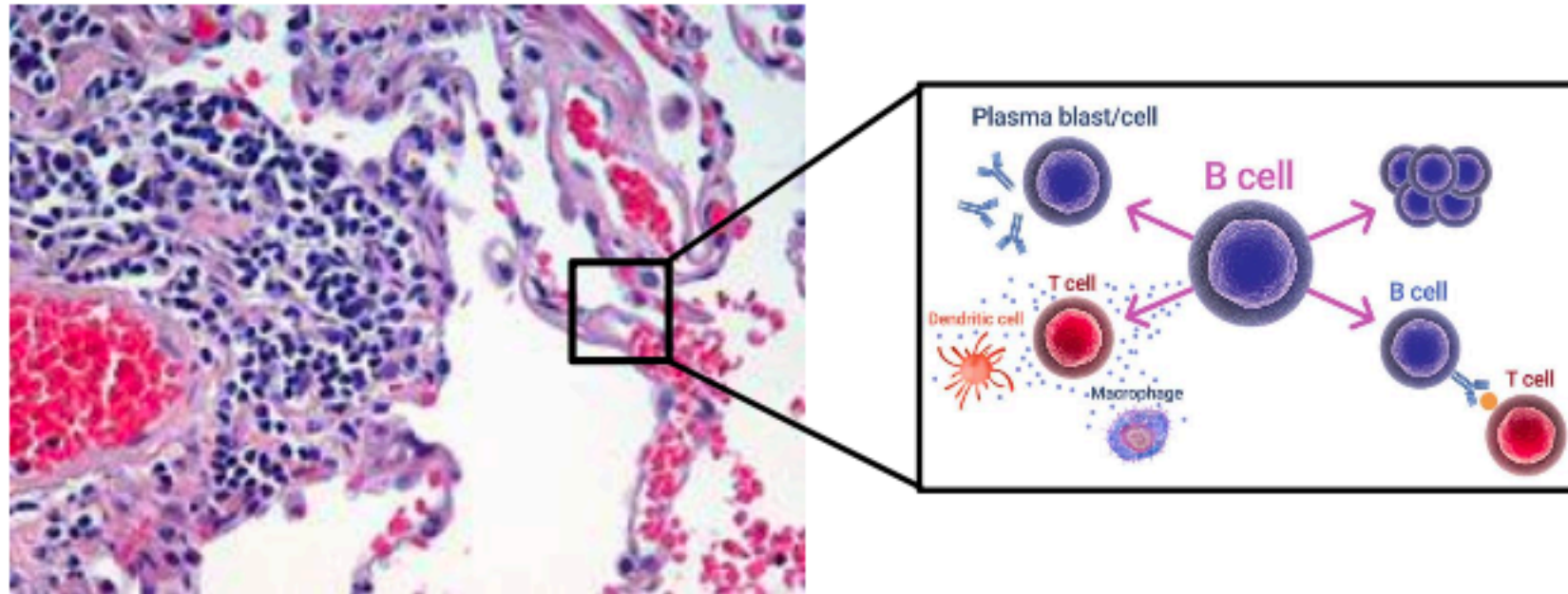


Network



Adjacency matrix

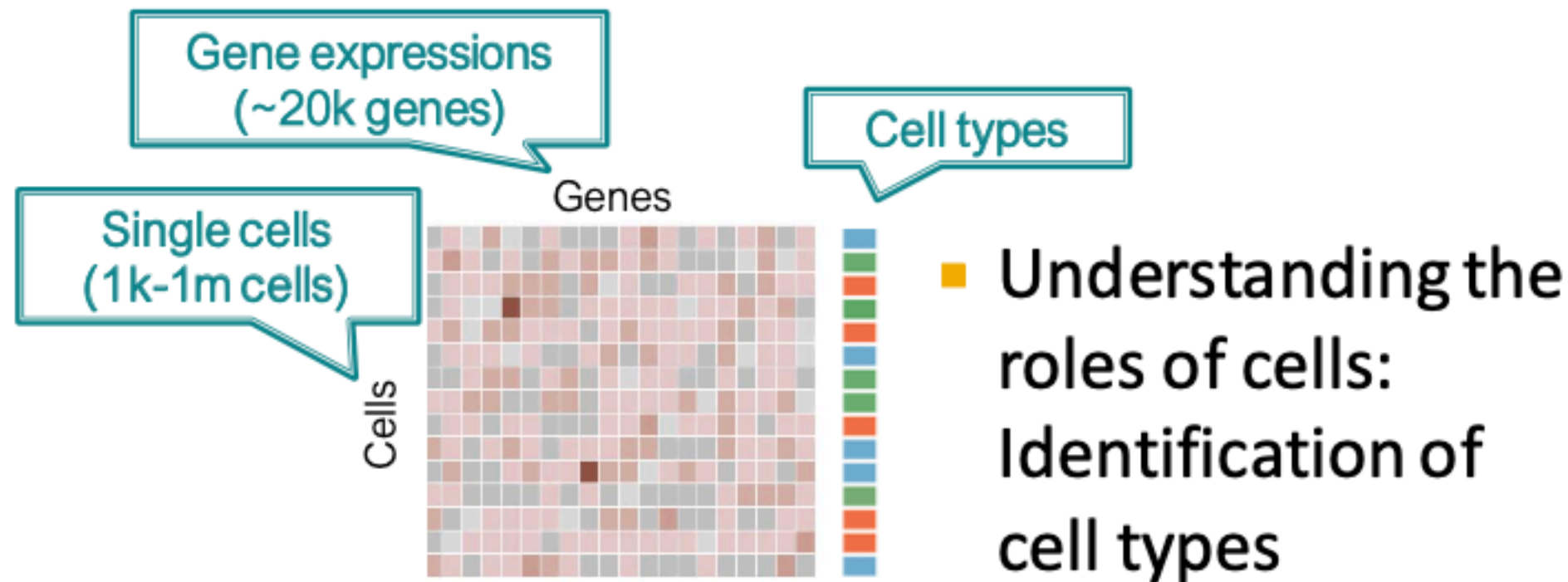
Every cell in a tissue has a specific role



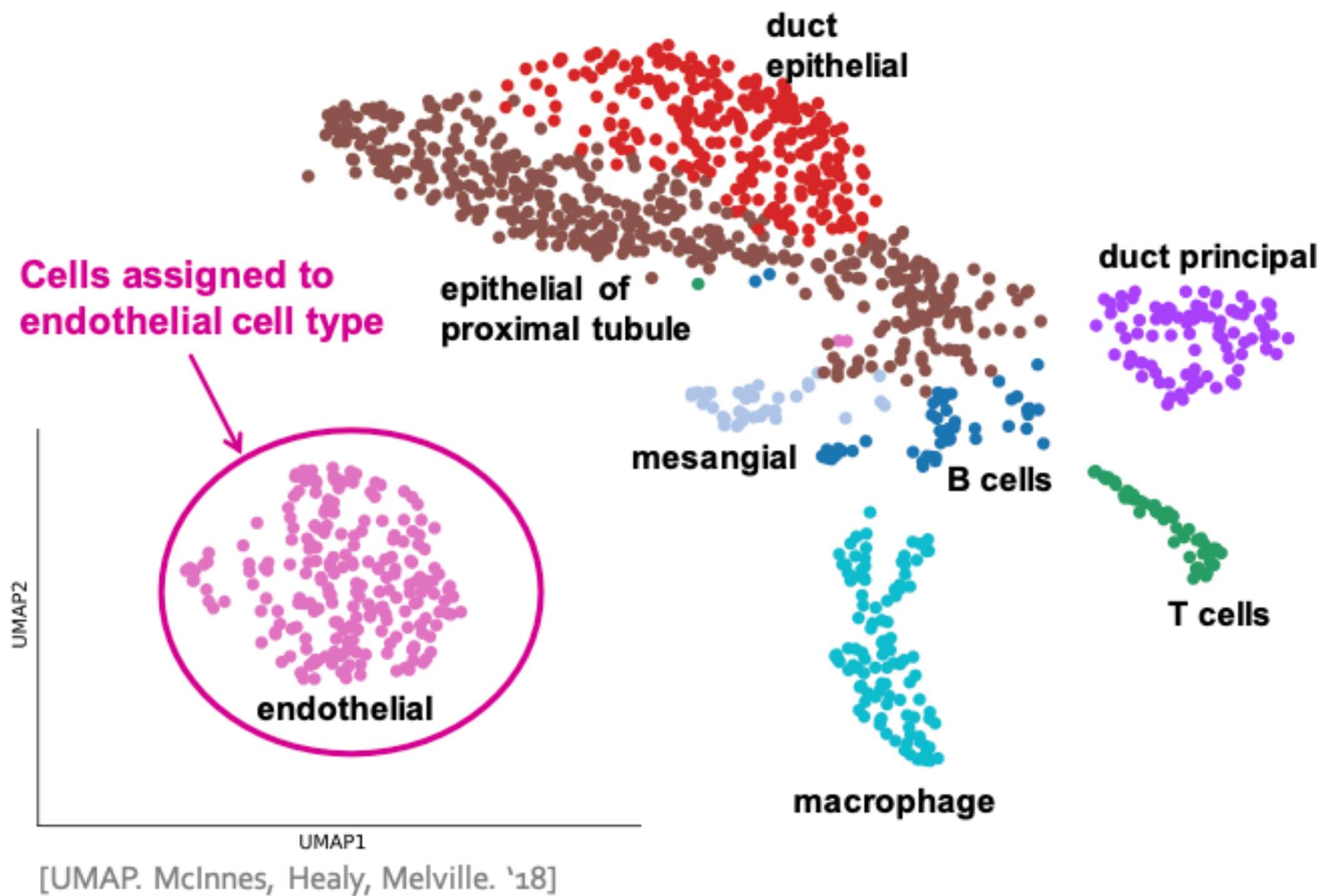
Challenge:

How to determine roles of cells?

Cell Type Identification Task



- **Cell type identification task: Given gene expressions of cells, assign cells to cell types**
 - Boils down to a **clustering task**: group cells according to their gene expression similarities



Challenges with Standard Clustering

- **Can we use standard clustering methods such as K-means to solve this problem?**
- **Why standard cluster methods do not work well?**
 - Data is very high-dimensional (~20k genes per cell)
 - Data is noisy and sparse (most values will be zero)
 - Number of clusters (cell types) is unknown
 - Cell types are hierarchically organized
 - **Definition of cell type is provisional**
 - **One cell type can have multiple cell subtypes**
 - **Where to put a threshold on a definition of a cell type?**

Idea: Represent Cells as a Graph

- **Idea:** Construct a **graph** between data points (cells) and detect **hierarchical network communities** in a graph

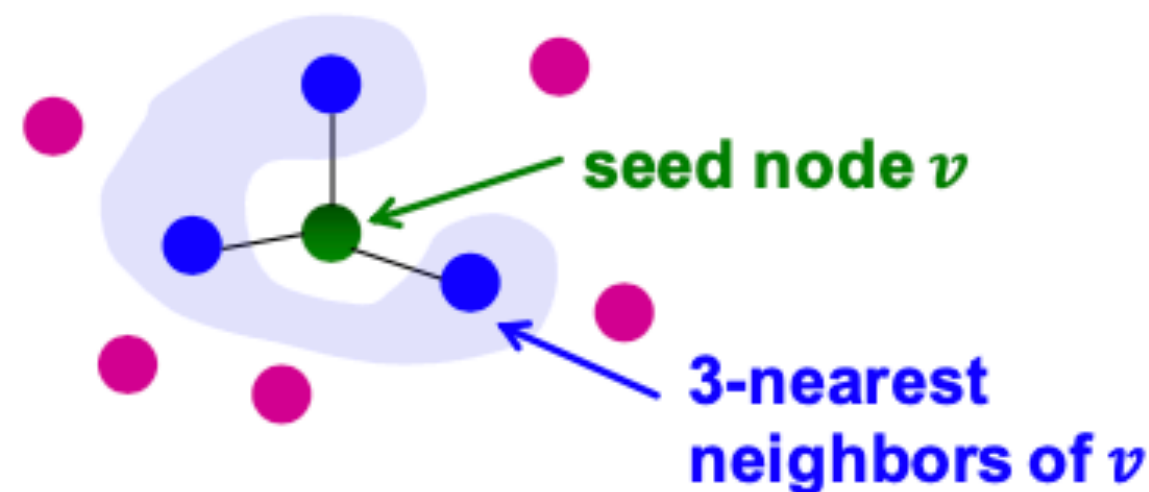
Why is graph a good representation?

- **Natural representation:** models cell-cell **interaction**
- Cells with more similar gene expressions are more likely to interact
 - Construct a graph based on similarities between gene expressions of cells
- Hierarchical network communities model well cell type hierarchy

We will cover next:

- 1) How to construct a graph from high-dimensional data?**
 - **Efficient k-NN graph construction**
- 2) How to define network communities?**
 - **Modularity**
- 3) How to detect communities?**
 - **Louvain algorithm**

- **K nearest neighbor (K-NN) graph:** Directed graph with vertex set V and an edge from each $v \in V$ to its **K most similar objects** in V under a given similarity measure
 - *E.g.*, cosine similarity, l_2 distance, l_1 distance



Example: $K=2$

Neighbors:

$$B[s] = \{c, d\}$$

Reverse neighbors:

$$R[s] = \{b, c, e\}$$

General neighbors:

$$B^*[s] = \{b, c, d, e\}$$

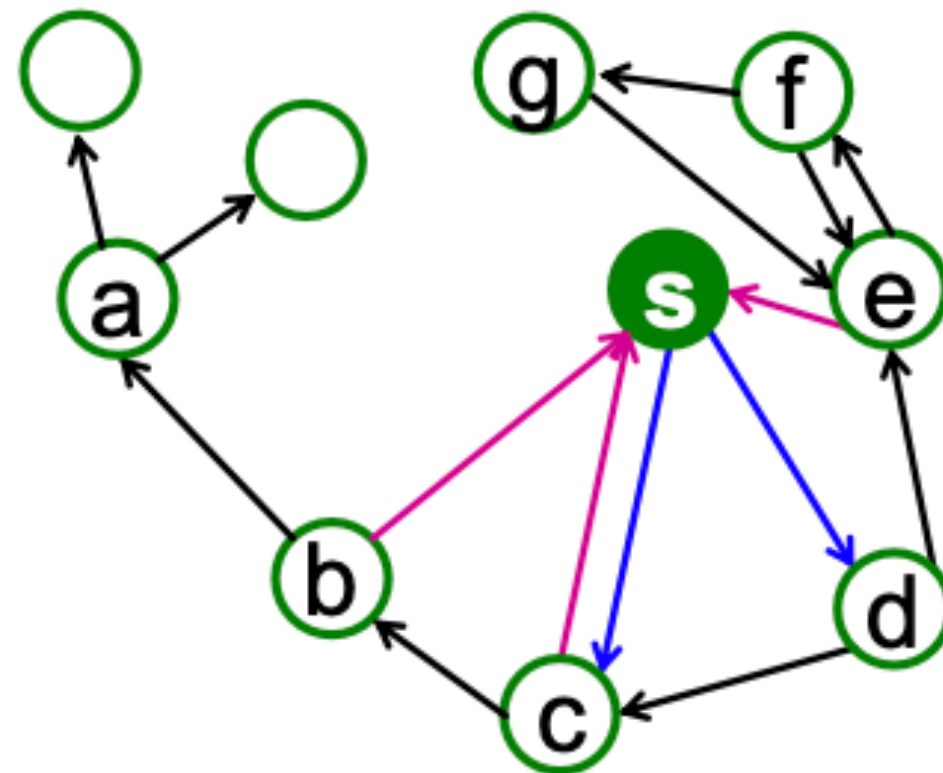
$$B^*[b] = \{a, c, s\}$$

$$B^*[c] = \{b, d, s\}$$

$$B^*[d] = \{c, e, s\}$$

$$B^*[e] = \{d, f, g, s\}$$

General neighbors of $B^*[s]$
are new candidates for $B[s]$



We will check $\{a, b, e, f, g\}$ as next candidates for $B[s]$:

Compute $\sigma(s, a)$, $\sigma(s, b)$, $\sigma(s, e)$, $\sigma(s, f)$, $\sigma(s, g)$ and update NNs of s

Arrows denote neighbors of a particular node. For example, arrow from b to s means that b selected s as its neighbor (but the opposite does not need to be true).

Further Improvements

- **Basic algorithm can be further optimized:**
 - **Local join:** Given node v and its neighbors $\bar{B}[v]$, compute similarity between each pair of different $p, q \in B^*[v]$ and update $B[p]$ and $B[q]$ with the similarity (reduces number of comparisons)
 - **Incremental search:** Attach a Boolean flag to remember if the objects have already been compared
 - **Early termination:** Count the number of K-NN list updates in each iteration, and stop when it becomes less than δKN where δ is a precision parameter
 - **MapReduce** implementation

Details in [Dong, Charikar, Li. *Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures*, 2011]

Cell Type Identification Example

- Which similarity measure σ to use?

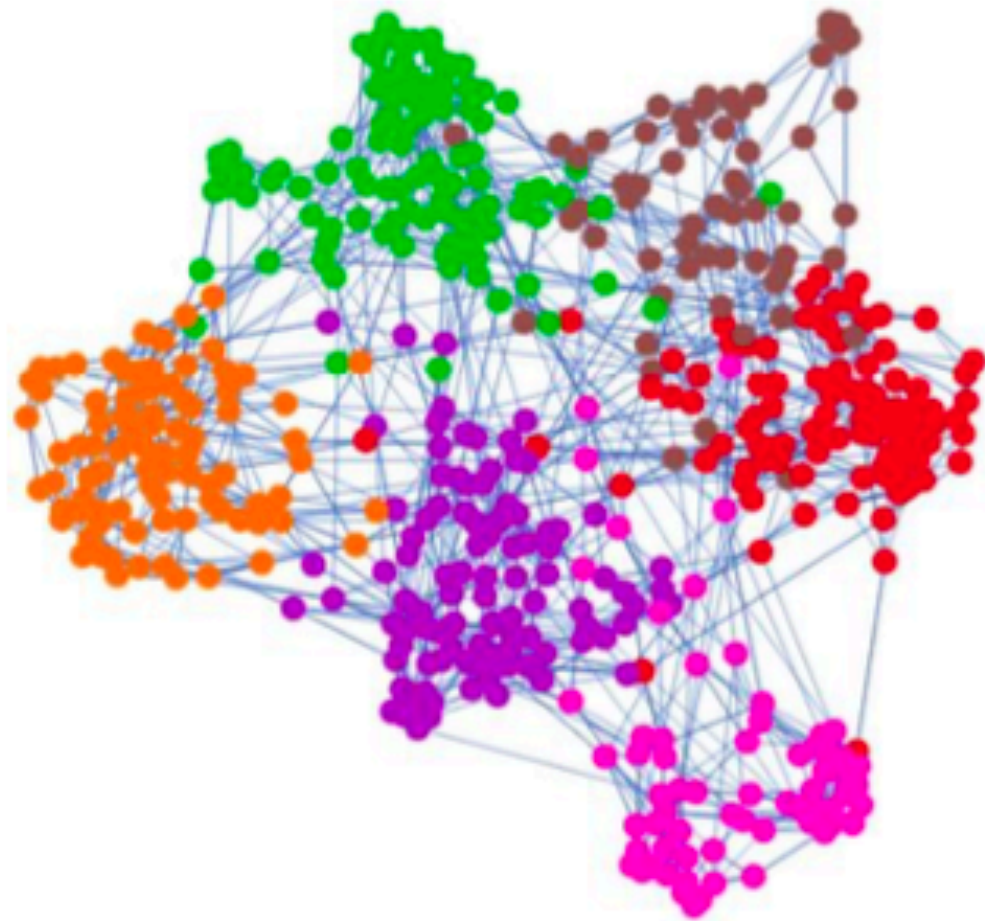
- Cells are compared based on their gene expression profiles
- **Challenge:** Number of genes is very high-dimensional



- **Approach:** First **apply SVD** (around 50 dimensions) and then compute l_2 distance in the low-dimensional space

After KNN Graph Construction

- Once we created K-NN graph of cells, how do we **define and detect network communities**?



Network Communities

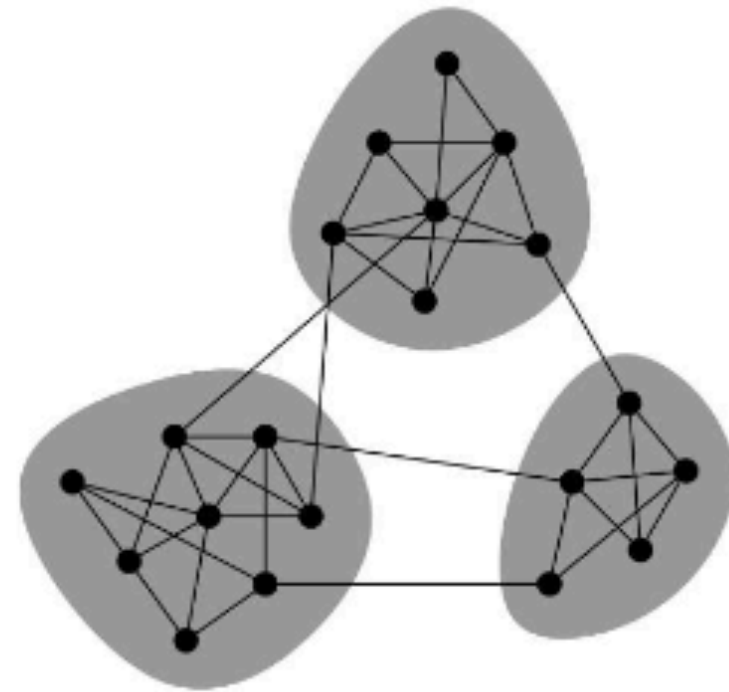
- **Communities:** sets of tightly connected nodes

- Define: **Modularity Q**

- A measure of how well a network is partitioned into communities
- Given a partitioning of the network into groups $s \in S$:

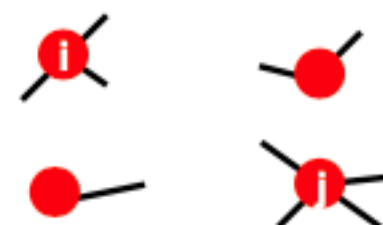
$$Q \propto \sum_{s \in S} [\text{\# edges within group } s) - \underbrace{(\text{expected \# edges within group } s)}]$$

Need a null model!



- Given real G on n nodes and m edges, construct rewired network G'

- Same degree distribution but random connections



- Consider G' as a **multigraph**

- The expected number of edges between nodes

i and j of degrees k_i and k_j equals to: $k_i \cdot \frac{k_j}{2m} = \frac{k_i k_j}{2m}$

- The expected number of edges in (multigraph) G' :

$$= \frac{1}{2} \sum_{i \in N} \sum_{j \in N} \frac{k_i k_j}{2m} = \frac{1}{2} \cdot \frac{1}{2m} \sum_{i \in N} k_i \left(\sum_{j \in N} k_j \right) =$$

$$= \frac{1}{4m} 2m \cdot 2m = m$$

Note:

$$\sum_{u \in V} k_u = 2m$$

- **Modularity of partitioning S of graph G:**

- $Q \propto \sum_{s \in S} [(\# \text{ edges within group } s) - (\text{expected } \# \text{ edges within group } s)]$

- $Q(G, S) = \underbrace{\frac{1}{2m}}_{\text{Normalizing const.: } -1 < Q < 1} \sum_{s \in S} \sum_{i \in s} \sum_{j \in s} \left(A_{ij} - \frac{k_i k_j}{2m} \right)$

Normalizing const.: $-1 < Q < 1$

$A_{ij} = 1$ if $i \rightarrow j$,
0 else

- **Modularity values take range $[-1, 1]$**

- It is positive if the number of edges within groups exceeds the expected number
- **Q greater than 0.3-0.7 means significant community structure**

$$Q(G, S) = \frac{1}{2m} \sum_{s \in S} \sum_{i \in s} \sum_{j \in s} \left(A_{ij} - \frac{k_i k_j}{2m} \right)$$

Equivalently modularity can be written as:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

- A_{ij} represents the edge weight between nodes i and j ;
- k_i and k_j are the sum of the weights of the edges attached to nodes i and j , respectively;
- $2m$ is the sum of all of the edge weights in the graph;
- c_i and c_j are the communities of the nodes; and
- δ is an indicator function

Idea: We can identify communities by maximizing modularity

Modularity Q ,

$$Q = \sum_{c \in C} \left[\frac{\Sigma_{\text{in}}^c}{2m} - \left(\frac{\Sigma_{\text{tot}}^c}{2m} \right)^2 \right]$$

Σ_{in}^c — the sum of the weights from all internal edges of community c ,

$$\Sigma_{\text{in}}^c = \sum_{\substack{u,v \in c \\ e(u,v) \in E}} w_{u,v}$$



Σ_{tot}^c — the sum of the weights from edges incident to any vertex in c ,

$$\Sigma_{\text{tot}}^c = \sum_{\substack{u \in c \text{ or } v \in c \\ e(u,v) \in E}} w_{u,v}$$



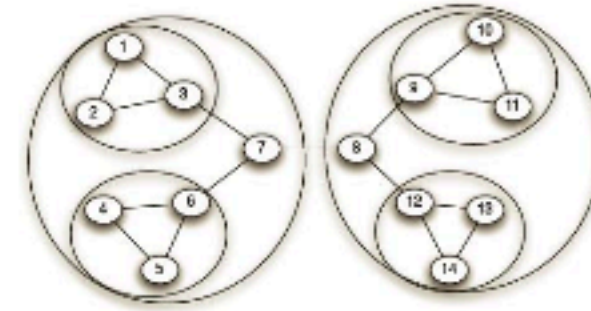
m — normalization factor, the sum of the weights across the graph,

$$m = \sum_{e(u,v) \in E} w_{u,v}$$

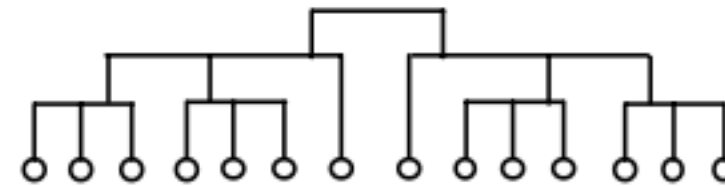
Louvain Algorithm

- **Greedy algorithm** for community detection
 - $O(n \log n)$ run time
- Supports weighted graphs
- Provides hierarchical communities
- Widely utilized to **study large networks** because:
 - Fast
 - Rapid convergence
 - High modularity output (i.e., “better communities”)

Network and communities:



Dendrogram:

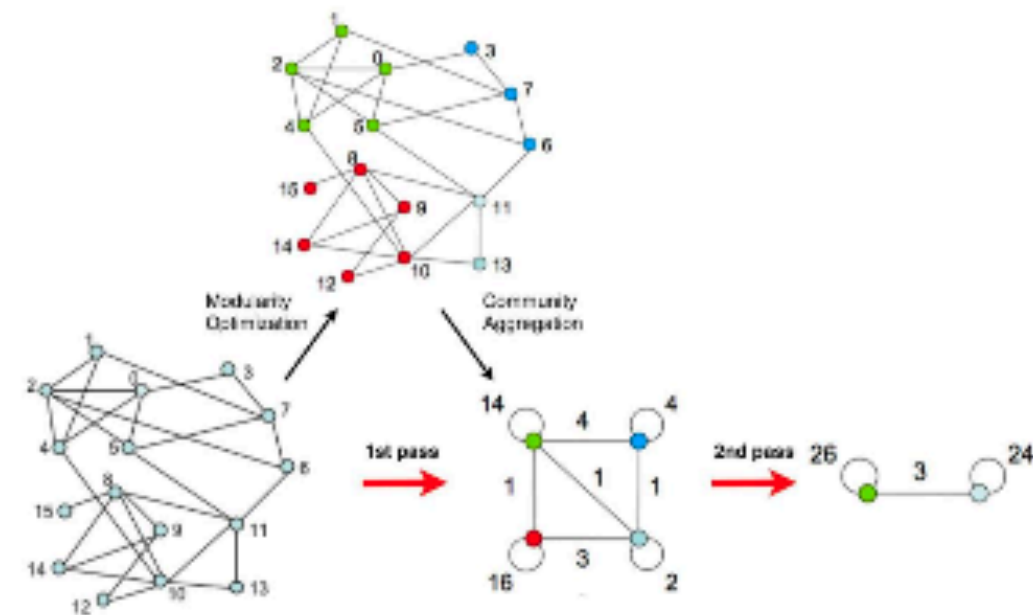


Lina Lackner & Mina Ghachami, Stanford CS223, "Fast unfolding of communities in large networks" Blondel et al. (2008)

Louvain Algorithm: At High Level

- Louvain algorithm **greedily maximizes** modularity
- **Each pass is made of 2 phases:**
 - **Phase 1:** Modularity is **optimized** by allowing only local changes to node-communities memberships
 - **Phase 2:** The identified communities are **aggregated** into super-nodes to build a new network
- **Goto Phase 1**

The passes are repeated **iteratively** until no increase of modularity is possible.



Louvain: 1st phase (Partitioning)

- Put each node in a graph into a **distinct community** (one node per community)
- For each node i , the algorithm performs two calculations:
 - Compute the modularity delta (ΔQ) when putting node i into the community of some neighbor j
 - Move i to a community of node j that yields the largest gain in ΔQ
- **Phase 1 runs until no movement yields a gain**

This first phase stops when a local maxima of the modularity is attained, i.e., when no individual node move can improve the modularity.
Note that the output of the algorithm depends on the order in which the nodes are considered.
Research indicates that the ordering of the nodes does not have a significant influence on the overall modularity that is obtained.

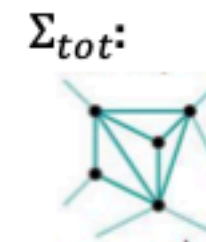
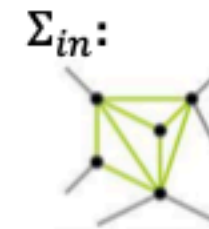
Louvain: Modularity Gain

What is ΔQ if we move node i to community C ?

$$\Delta Q(i \rightarrow C) = \left[\frac{\Sigma_{in} + k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right]$$

■ where:

- Σ_{in} ... sum of link weights between nodes in C
- Σ_{tot} ... sum of all link weights of nodes in C
- $k_{i,in}$... sum of link weights between node i and C
- k_i ... sum of all link weights (i.e., degree) of node i



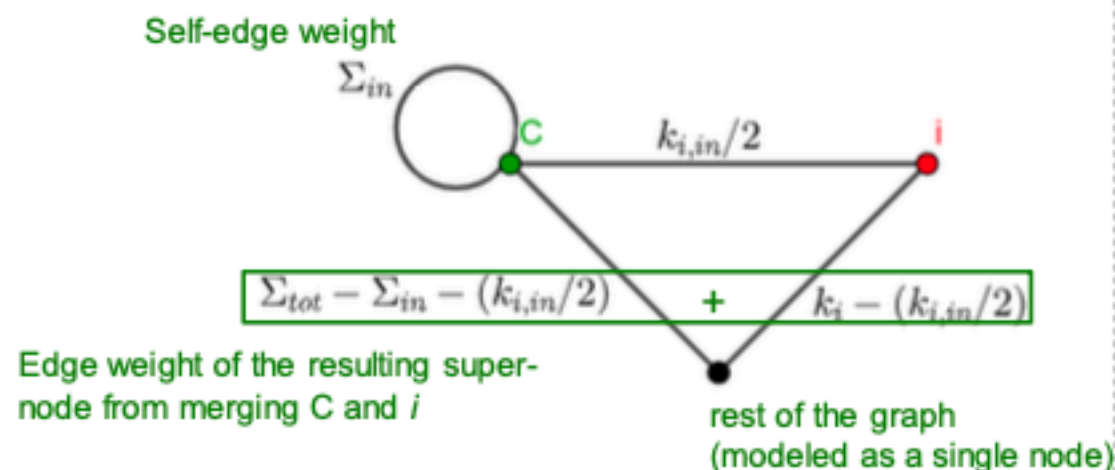
Louvain: Modularity Gain

More in detail:

Modularity contribution
after merging node i

Modularity contribution
before merging node i

$$\Delta Q(i \rightarrow C) = \left[\frac{\Sigma_{in} + k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\underbrace{\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2}_{\text{Modularity of } C} - \underbrace{\left(\frac{k_i}{2m} \right)^2}_{\text{Modularity of } i} \right]$$



By applying the Modularity definition:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Louvain: Modularity Gain

- What is ΔQ if we move node i to community C ?

$$\Delta Q(i \rightarrow C) = \left[\frac{\sum_{in} + k_{i,in}}{2m} - \left(\frac{\sum_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right]$$

- Also need to derive $\Delta Q(D \rightarrow i)$ of taking node i out of community D .
- And then: $\Delta Q = \Delta Q(i \rightarrow C) + \Delta Q(D \rightarrow i)$

Louvain: 2nd phase (Restructuring)

- The communities obtained in the first phase are contracted into **super-nodes**, and the network is created accordingly:
 - Super-nodes are connected if there is at least one edge between the nodes of the corresponding communities
 - The weight of the edge between the two super-nodes is the sum of the weights from all edges between their corresponding communities
- **Phase 1 is then run on the super-node network**

Louvain Algorithm

Algorithm 1: Sequential Louvain Algorithm

Input: $G=(V,E)$: graph representation.

Output: C : community sets at each level;

Q : modularity at each level.

Var: $\hat{c}$: vertex u 's best candidate community set.

```

1 Loop outer
2    $C \leftarrow \{\{u\}\}, \forall u \in V$ ;
3    $\Sigma_{in}^c \leftarrow \sum w_{u,v}, e(u,v) \in E, u \in c \text{ and } v \in c$ ;
4    $\Sigma_{tot}^c \leftarrow \sum w_{u,v}, e(u,v) \in E, u \in c \text{ or } v \in c$ ;
5   // Phase 1.
6   Loop inner
7     for  $u \in V$  and  $u \in c$  do
8       // Find the best community for vertex u.
9        $\hat{c} \leftarrow \underset{\forall c', \exists e(u,v) \in E, v \in c'}{\operatorname{argmax}} \Delta Q_{u \rightarrow c'};$  Modularity gain
10      if  $\Delta Q_{u \rightarrow \hat{c}} > 0$  then
11        // Update  $\Sigma_{tot}$  and  $\Sigma_{in}$ .
12         $\Sigma_{tot}^{\hat{c}} \leftarrow \Sigma_{tot}^{\hat{c}} + w(u)$ ;  $\Sigma_{in}^{\hat{c}} \leftarrow \Sigma_{in}^{\hat{c}} + w_{u \rightarrow \hat{c}}$ ;
13         $\Sigma_{tot}^c \leftarrow \Sigma_{tot}^c - w(u)$ ;  $\Sigma_{in}^c \leftarrow \Sigma_{in}^c - w_{u \rightarrow c}$ ;
14        // Update the community information.
15         $\hat{c} \leftarrow \hat{c} \cup \{u\}$ ;  $c \leftarrow c - \{u\}$ ;
16      if No vertex moves to a new community then
17        exit inner Loop;

```

Halting criterion for 1st Phase

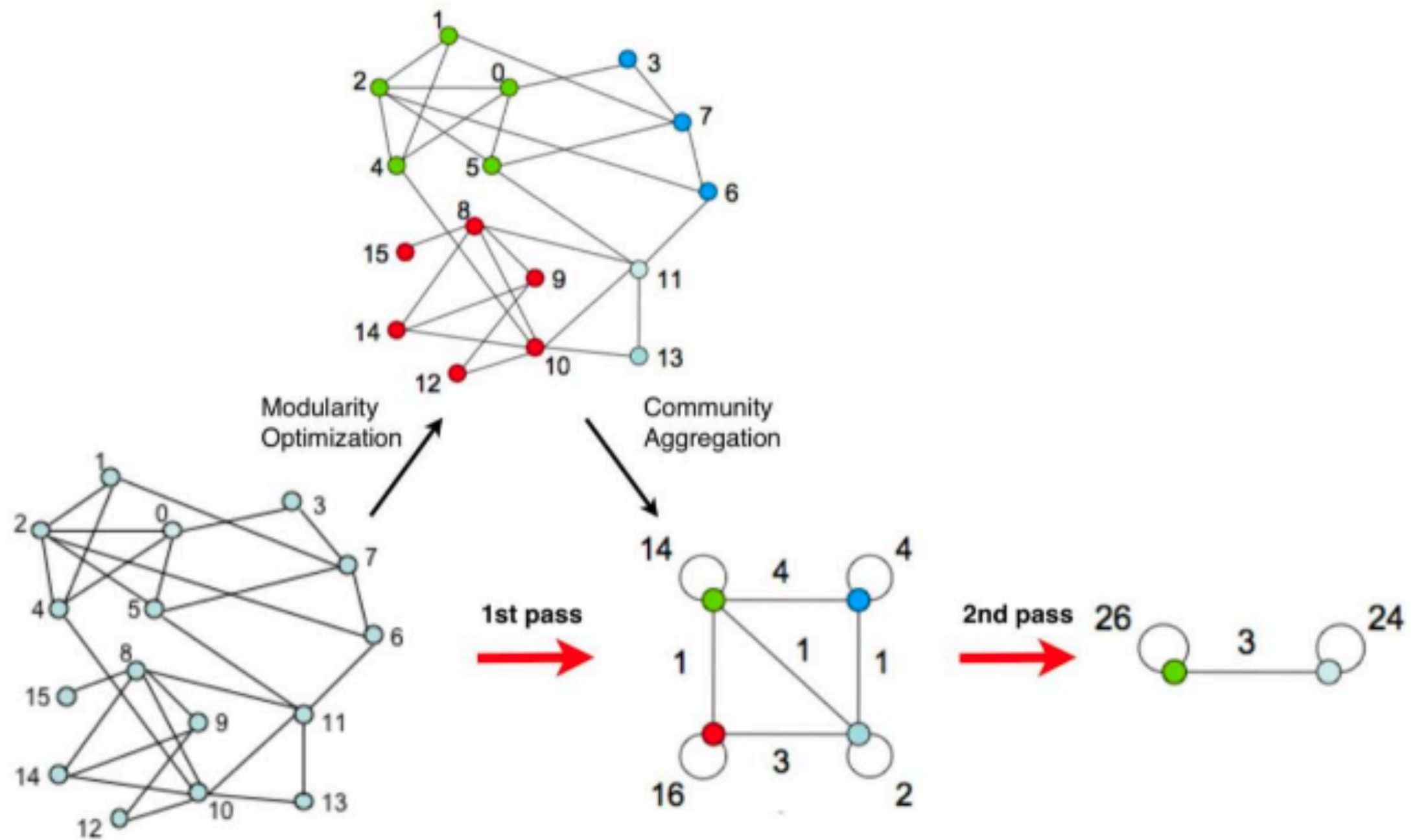
```

18 // Calculate community set and modularity.
19  $Q \leftarrow 0$ ;
20 for  $c \in C$  do
21    $Q \leftarrow Q + \frac{\Sigma_{in}^c}{2m} - (\frac{\Sigma_{tot}^c}{2m})^2$ ;
22  $C' \leftarrow \{c\}, \forall c \in C$ ; print  $C'$  and  $Q$ ;
23 // Phase 2: Rebuild Graph.
24  $V' \leftarrow C'$ ; Communities contracted into super-nodes
25  $E' \leftarrow \{e(c, c')\}, \exists e(u, v) \in E, u \in c, v \in c'$ ;
26  $w_{c, c'} \leftarrow \sum w_{u, v}, \forall e(u, v) \in E, u \in c, v \in c'$ ;
27 if No community changes then
28   exit outer Loop;
29  $V \leftarrow V'$ ;  $E \leftarrow E'$ ; Halting criterion for 2nd Phase

```

the weights of the edges between the new super-nodes are given by the **sum of the weights of the edges** between vertices in the corresponding two communities

Louvain Algorithm



Back to Detecting Cell Types

Input:

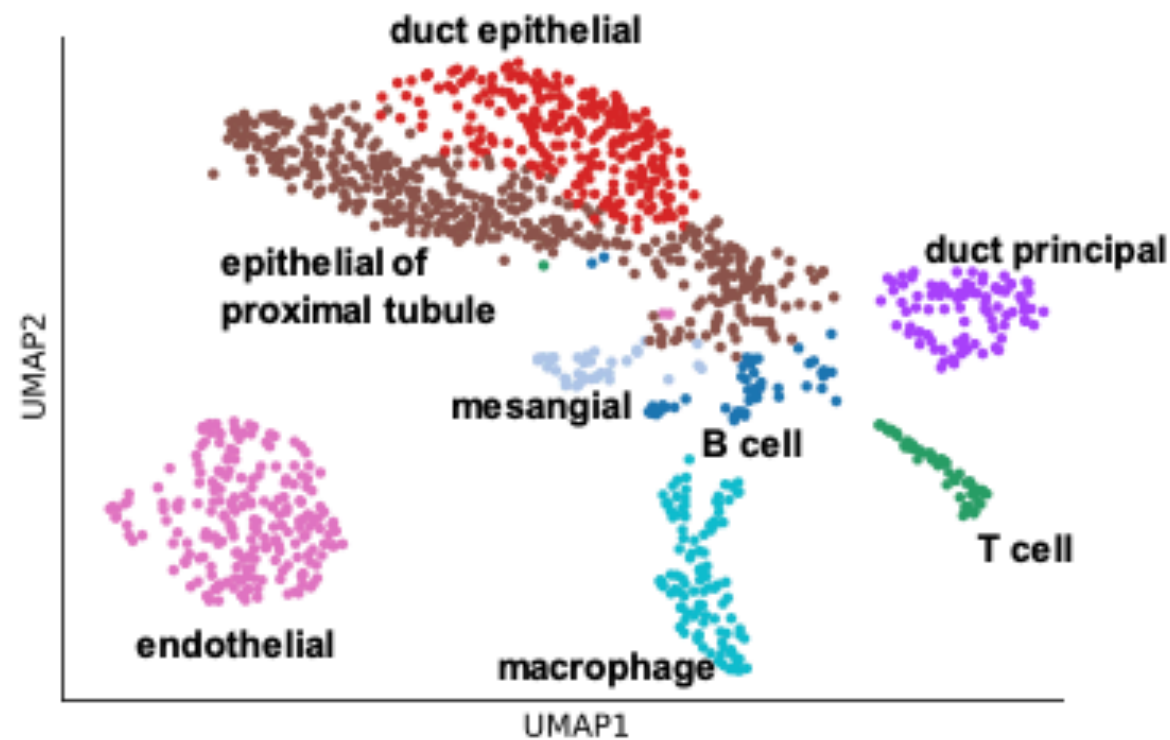
Single-cell gene expression data

Steps:

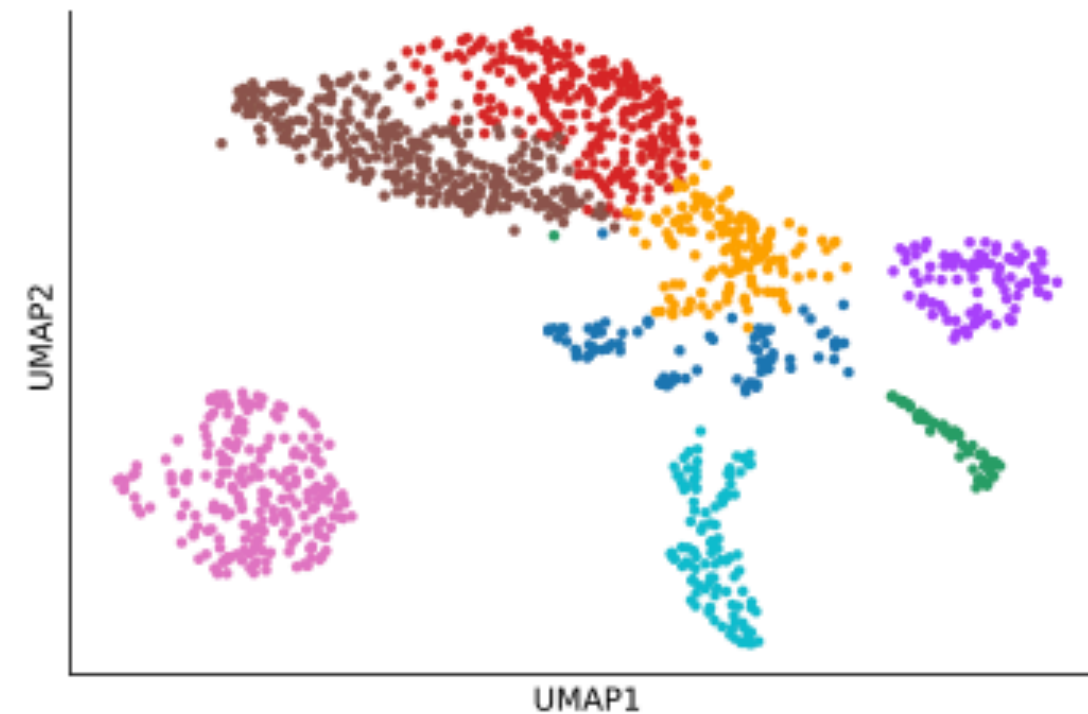
- 1) Apply SVD to cell gene expression data (~50 dim)
- 2) Create K-NN (K=15) graph between the low-dim cell gene expressions
- 2) Apply the Louvain algorithm to identify the clusters

Cell Type Identification Task

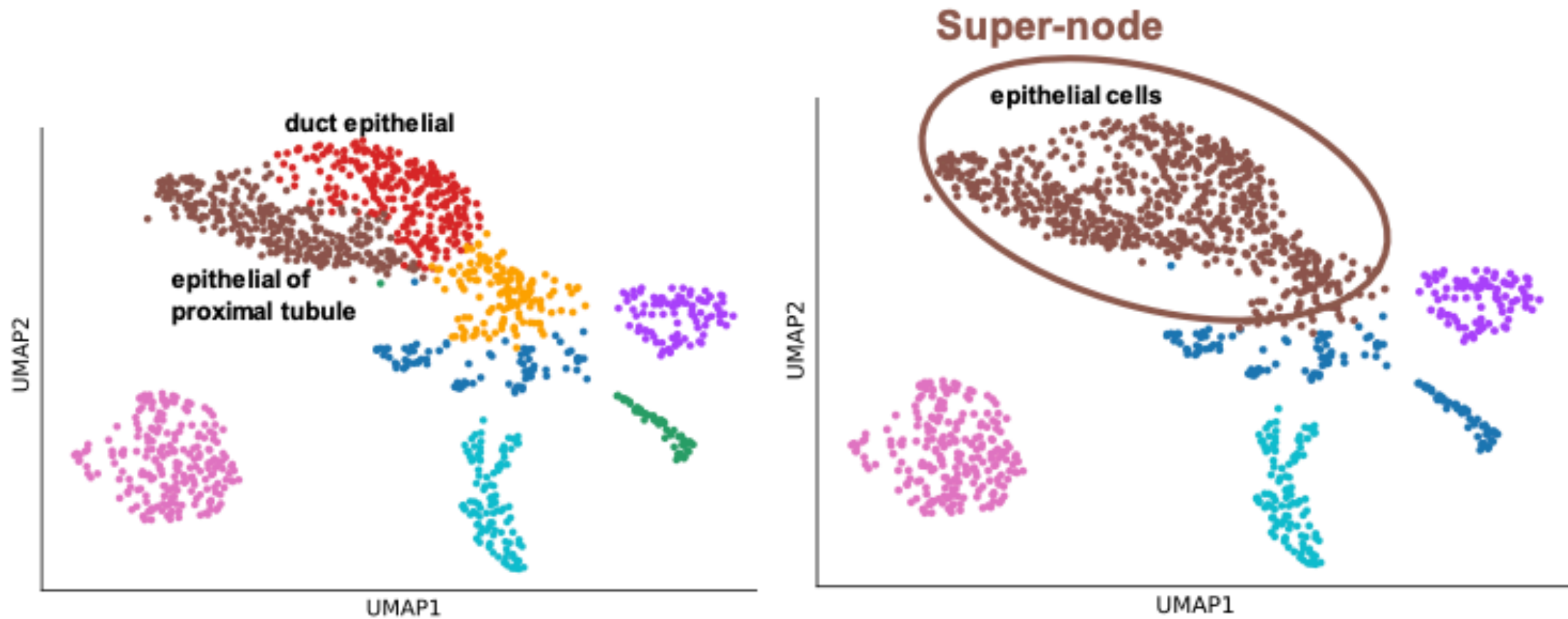
Ground truth annotations



Louvain algorithm



Louvain Hierarchical Groups



Summary: Modularity

- **Modularity:**

- Overall quality of the partitioning of a graph into communities
- Used to determine the number of communities

- **Louvain modularity maximization:**

- Greedy strategy
- Great performance, scales to large networks

Further reading

- Network Motif
- Graph Laplacian and spectral clustering
- Network alignment
 - Local alignment
 - Global alignment
 - Conserved network



THANK YOU